

Объектно-ориентированное программирование

это подход, при котором программа рассматривается как набор объектов, взаимодействующих друг с другом. У каждого есть свойства и поведение. Если постараться объяснить простыми словами, то ООП ускоряет написание кода и делает его более читаемым.

Идеология объектно-ориентированного программирования (ООП) разрабатывалась, чтобы связать поведение определенного объекта с его классом. Людям проще воспринимать окружающий мир как объекты, которые поддаются определенной классификации (например, разделение на живую и неживую природу).

Объектно-ориентированное программирование.....	1
Практическое задание № 1.....	4
Теоретическая часть.....	4
Практическая часть.....	6
Задание.....	10
Контрольные вопросы.....	10
Практическое задание № 2.....	11
Теоретическая часть.....	11
Практическая часть.....	14
Задание.....	16
Контрольные вопросы.....	16
Практическое задание № 3.....	17
Теоретическая часть.....	17
Практическая часть.....	20
Задание.....	22
Контрольные вопросы.....	22
Практическое задание № 4.....	23
Теоретическая часть.....	23
Практическая часть.....	26
Задание.....	27
Контрольные вопросы.....	27
Практическое задание № 5.....	28
Теоретическая часть.....	28
Практическая часть.....	30
Задание.....	31
Контрольные вопросы.....	32
Практическое задание № 6.....	32
Теоретическая часть.....	32
Практическая часть.....	35
Задание.....	36
Контрольные вопросы.....	36
Практическое задание № 7.....	37
Теоретическая часть.....	37
Практическая часть.....	39
Задание.....	41
Контрольные вопросы.....	41
Практическое задание № 8.....	42
Теоретическая часть.....	42
Практическая часть.....	45
Задание.....	48
Контрольные вопросы.....	48
Практическое задание № 9.....	48
Теоретическая часть.....	49

Задание.....	55
Контрольные вопросы.....	55
Практическое задание № 10.....	56
Теоретическая часть.....	56
Практическая часть.....	61
Задание.....	63
Контрольные вопросы.....	63
Практическое задание № 11.....	64
Теоретическая часть.....	64
Практическая часть.....	66
Задание.....	69
Контрольные вопросы.....	69
Практическое задание № 12.....	70
Теоретическая часть.....	70
Практическая часть.....	71
Задание.....	72
Контрольные вопросы.....	72
Практическое задание № 13.....	73
Теоретическая часть.....	73
Практическая часть.....	75
Задание.....	78
Контрольные вопросы.....	78
Практическое задание № 14.....	79
Теоретическая часть.....	79
Практическая часть.....	81
Задание.....	82
Контрольные вопросы.....	82
Практическое задание № 15.....	83
Теоретическая часть.....	83
Практическая часть.....	84
Самостоятельная работа №1.....	86
Материал для ознакомления:.....	86
Задание.....	86
Самостоятельная работа №2.....	87
Материал для ознакомления:.....	87
Задание:.....	87
Самостоятельная работа №3.....	88
Материал для ознакомления:.....	88
Контрольные вопросы:.....	88

Практическое задание № 1

Принципы ооп

Теоретическая часть

Инкапсуляция

Инкапсуляция означает скрытие деталей реализации объекта и предоставление только интерфейса для взаимодействия с ним. Это позволяет изолировать изменения в одной части программы от других частей, что делает код более надежным и устойчивым к изменениям.

Наследование

Наследование позволяет создавать новые классы на основе существующих. Это способствует повторному использованию кода и созданию иерархий классов. Наследование позволяет наследникам использовать свойства и методы предков и переопределять или расширять их, если это необходимо.

Полиморфизм

Полиморфизм означает способность объектов разных классов обладать общим интерфейсом. Это позволяет обрабатывать объекты разных типов с помощью общих методов и функций. Полиморфизм делает код более гибким и расширяемым.

Абстракция

Абстракция - это процесс выделения общих характеристик объектов и создание абстрактных классов или интерфейсов для их представления.

Абстракция помогает упростить модель системы, делая её более понятной и управляемой.

Пример:



```
1 class Animal:
2     def __init__(self, name):
3         self.name = name
4
5     def speak(self):
6         pass
7
8 class Dog(Animal):
9     def speak(self):
10         return f"{self.name} говорит: Гав!"
11
12 class Cat(Animal):
13     def speak(self):
14         return f"{self.name} говорит: Мяу!"
15
16 # Создаем объекты
17 dog = Dog("Барсик")
18 cat = Cat("Мурзик")
19
20 # Вызываем метод speak() для объектов разных классов
21 print(dog.speak()) # Вывод: "Барсик говорит: Гав!"
22 print(cat.speak()) # Вывод: "Мурзик говорит: Мяу!"
```

Практическая часть

Для начало надо определиться, чем отличаются объекты от класса. Класс это схема, при заполнений этой схемы, создается объект класса. Например, есть класса Human с атрибутами класса `рост(height)`, `возраст(age)`:



Изначально атрибуты класса `height`, `age` со значением `None`. При создание объекта класса они будут наследоваться. Создаем объекты класса `human_one`, `human_two`.



```
1 class Human:
2     height = None
3     age = None
4
5
6 human_one = Human()
7 human_two = Human()
```

И проверим значение наших атрибутов класса в объекте класса.



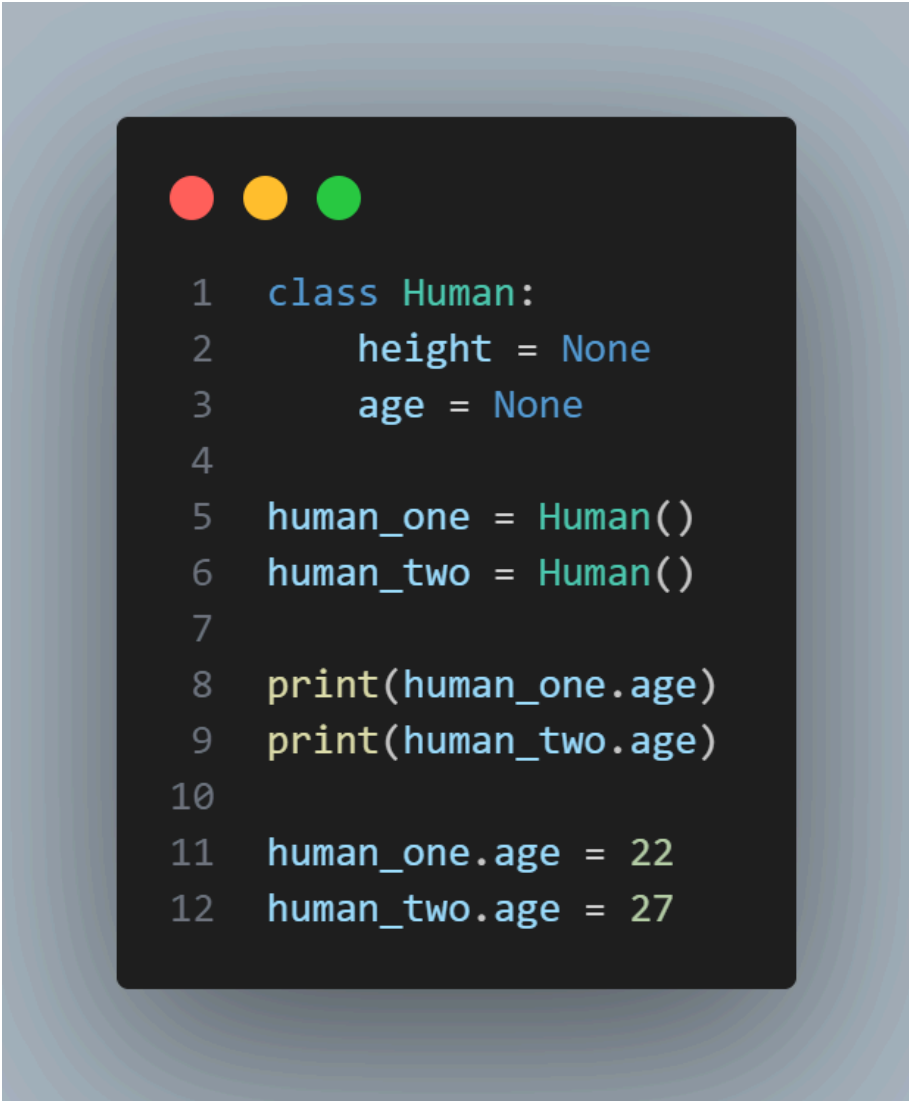
```
1 class Human:
2     height = None
3     age = None
4
5 human_one = Human()
6 human_two = Human()
7
8 print(human_one.age)
9 print(human_two.age)
```

При запуске программы, она будет выводить значение атрибута класса в объекте –

None

None

Также, если в атрибуте класса ничего нету, то не инициализировался (и его как бы не существует)
Присвоим другое значение атрибута класса в объекте. Для это обращаемся к объекту с атрибутами
класса(как выводили атрибут класса) и присваиваем другое значение.



```
1  class Human:
2      height = None
3      age = None
4
5  human_one = Human()
6  human_two = Human()
7
8  print(human_one.age)
9  print(human_two.age)
10
11 human_one.age = 22
12 human_two.age = 27
```

Теперь в атрибуте объекте будут храниться другие значение. Также давайте удалим атрибут
класса height для этого воспользуемся функцией `delattr(obj, name_attr)`, которая принимает:

Первым аргументам: объект класса или сам класс

Вторым аргументам: название атрибута в виде строки

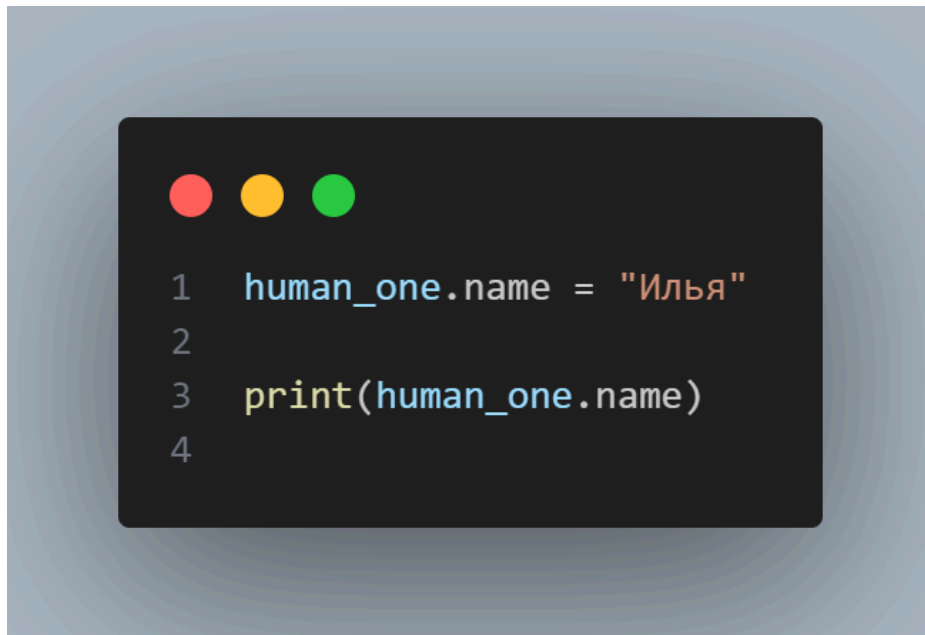
Удаляем атрибут height у обоих объектов, и с помощью функции `hasattr(obj, name_attr)`
проверим существует ли данный атрибут в объекте. При вызове данной функции она возвращает
bool значение.



```
1 class Human:
2     height = None
3     age = None
4
5 human_one = Human()
6 human_two = Human()
7
8 print(human_one.age)
9 print(human_two.age)
10
11 human_one.age = 22
12 human_two.age = 27
13
14 print(human_one.age)
15 print(human_two.age)
16
17 delattr(human_one, 'height')
18 delattr(human_two, 'height')
19
20
21 print(hasattr(human_one, 'height'))
22 print(hasattr(human_two, 'height'))
```

Теперь при вызове функции `hasattr`, она будет возвращаться `False`, потому что данного атрибута нету.

Давайте добавим новый атрибут для объекта класса, например `name`. Это можно сделать с помощью функции `setattr` или обратиться к объекту класса, и через точку написать название атрибута, а также присвоить значение.



Задание

1. Создайте класс Goods с атрибутами класса и значениями:
 - a. title Мороженое
 - b. weight 151
 - c. tp "Еда"
 - d. price 12321Изменить значение атрибута price и weight
2. Создайте пустой класс Car, и добавьте атрибуты со значениями:
 - a. model Тойота
 - b. color черный
 - c. number П34А123
3. Что делает хранит атрибут `__dict__`? (самостоятельно узнать)

Контрольные вопросы

1. напишите функции, которые удаляют атрибут класса, создают атрибут класса, проверяют существует ли атрибут класса и возвращают значение атрибута класса
2. Чем отличается объект от класса ?

Практическое задание № 2

Методы. Параметр self

Теоретическая часть

Методы в Python

Метод — это функция, определенная внутри класса. Методы используются для того, чтобы объекты класса могли выполнять определенные действия или изменять свои внутренние данные (атрибуты). Все методы связаны с конкретным экземпляром класса и работают с его данными через параметр `self`.

Параметр `self`

`self` — это специальный параметр, который всегда передается первым в методы класса. Он является ссылкой на текущий экземпляр класса и используется для доступа к атрибутам и другим методам этого экземпляра. Это ключевой механизм, который позволяет методам "знать", с каким объектом они работают.

Основные моменты о `self`:

1. **Связь с объектом:** Через `self` метод может обращаться к атрибутам и методам конкретного экземпляра класса. Это необходимо, так как каждый объект класса может иметь свои значения атрибутов.
2. **Неявная передача:** Когда вызывается метод объекта, Python автоматически передает ссылку на этот объект как первый аргумент метода — именно в качестве `self`.
3. **Необязательное имя:** Хотя общепринятое имя для этого параметра — `self`, можно использовать любое другое, но следование стандарту помогает делать код понятнее.

Пример использования метода с self

```
1 class Dog:
2     # Инициализация объекта (без магического метода __init__)
3     def set_details(self, name, breed):
4         self.name = name    # Атрибут "name" объекта
5         self.breed = breed  # Атрибут "breed" объекта
6
7     # Метод для вывода информации об объекте
8     def display_details(self):
9         print(f"Dog's name: {self.name}, Breed: {self.breed}")
10
11 # Создание экземпляра класса Dog
12 my_dog = Dog()
13
14 # Вызов метода для установки атрибутов
15 my_dog.set_details("Buddy", "Golden Retriever")
16
17 # Вызов метода для отображения данных
18 my_dog.display_details() # Вывод: Dog's name: Buddy, Breed: Golden Retriever
```

Как работает self в примере:

1. Метод set_details:

- Использует self для установки атрибутов экземпляра (self.name и self.breed).
- Атрибуты объекта становятся доступными для других методов этого объекта.

2. Метод display_details:

- С помощью self получает доступ к атрибутам name и breed, установленных ранее, и выводит их.

Почему важно использовать self?

- **Связь метода с объектом:** Без self методы не могли бы взаимодействовать с состоянием конкретного объекта. Если бы методы не принимали self, они не смогли бы изменять и использовать данные конкретного экземпляра класса.
- **Индивидуальные объекты:** Каждый объект класса может иметь свои значения атрибутов, и через self методы работают с данными конкретного объекта, а не класса в целом.

Пример без использования self (ошибочный подход):

Если убрать self, код перестанет работать, так как Python не сможет определить, с какими именно атрибутами и методами объекта следует взаимодействовать.



```
1 class Dog:
2     # Инициализация объекта (без магического метода __init__)
3     def set_details(self, name, breed):
4         self.name = name    # Атрибут "name" объекта
5         self.breed = breed  # Атрибут "breed" объекта
6
7     # Метод для вывода информации об объекте
8     def display_details(self):
9         print(f"Dog's name: {self.name}, Breed: {self.breed}")
10
11 # Создание экземпляра класса Dog
12 my_dog = Dog()
13
14 # Вызов метода для установки атрибутов
15 my_dog.set_details("Buddy", "Golden Retriever")
16
17 # Вызов метода для отображения данных
18 my_dog.display_details() # Вывод: Dog's name: Buddy, Breed: Golden Retriever
```

Этот пример вызовет ошибку, так как без `self` Python не понимает, к каким атрибутам или методам обращаться.

Практическая часть

В этой практической сделаем два класса. Первый класс будет обрабатывать txt файл с атрибутами и значениями, а затем создавать новый класс с атрибутами и значениями из файла.

Создаем классы UserSchem и DataBase. Класс UserSchem создаем пустым используя оператор pass, в DataBase создаем методы(методы это такая же функция, но внутри класса) get_data, serializers и create_user.

1. метод get_data – возвращать объект файла в кодировке UTF-8
2. метод serializers – обрабатывает объект файла, и возвращает его виде списка со словарями
3. метод create – объекты класс с определенными атрибутами и значениями, которые взяты из файла

Структура txt файла:

```
id 1 name Илья age 23
id 2 name Михаил age 32
id 4 name Дмитрий age 25
```

Метод get_data:

```
1 def get_data(self, url):
2     with open(url, 'r', encoding='UTF-8') as f:
3         result = f.readlines()
4         f.close()
5     return result
```

В методе get_data – с помощью встроенной функции open для взаимодействия с файлами. Функция принимает путь файл, типа взаимодействия(в данном примере – чтение “r”), а также необязательный аргумент это тип кодировке(так как у нас кириллица, используем utf-8). Используем with...as, чтобы определенное значение использовать в некой области, без создания переменной. Используем метод, который возвращают все строки в файле.

Метод serializers:

```
1 def serializers(self, data:TextIOWrapper):
2     content = []
3     for i in data:
4         schema=dict()
5         line = [i for i in re.split(r'\s',i) if i != '']
6         for index in range(0,len(line)-1, 2):
7             schema[line[index]] = line[index+1]
8         content.append(schema)
9     return content
```

Обрабатывает данные из файла в виде словаря с помощью регулярных выражений.

метод create:

```
1 def create(self, data):
2     for i in data:
3         user = UserSchem()
4         for key, item in i.items():
5             setattr(user,key, item)
```

create создает объекты, но нигде не сохраняют. Как вообще сохраняются объекты? Объекты сохраняются, тогда когда они используются в коде.

Задание

1. Измените метод `create`, чтобы он сохранял все созданные объекты в виде списка в атрибуте класса
2. Добавьте метод `search`, который находит определенный объект по атрибутам
3. Создайте класс `Translator` с методами `add`, `remove`, `translate`.
 - `add(self, eng, rus)` – добавляет в словарь(dict) перевод английского слова на русский язык (если перевод слова на английском существует, то добавляет его в список со всеми переводами данного слова. Перевод в списке не должен повторяться). Также создам атрибут класса `tr`, где будет храниться словарь
 - `remove(self, eng)` – удаляет слова из словаря со списком перевода
 - `translate(self, eng)` – возвращает перевод слова с первым значением из списка

Контрольные вопросы

1. Что называется методом класса?
2. Какую роль играет параметр `self` в методах класса?

Практическое задание № 3

Магические методы.
Магический метод `__init__`.
Магический метод `__del__`.

Теоретическая часть

Магические методы в Python, также известные как dunder-методы (от английского "double underscore"), представляют собой специальные методы, которые начинаются и заканчиваются двумя подчеркиваниями (например, `__init__`, `__str__`). Эти методы позволяют вам определять поведение ваших объектов при использовании встроенных функций и операторов. Магические методы делают ваш код более читаемым и позволяют использовать объекты классов так, как если бы они были встроенными типами данных. Например, вы можете использовать магические методы для создания объектов, которые ведут себя как списки или словари, или для определения поведения объектов при выполнении арифметических операций.

Магические методы играют ключевую роль в Python, так как они позволяют разработчикам создавать более интуитивно понятные и гибкие классы. Они позволяют вам переопределять стандартное поведение объектов, что может быть особенно полезно при создании пользовательских структур данных или при работе с объектно-ориентированным программированием. В этой статье мы рассмотрим основные магические методы, их назначение и примеры использования, чтобы вы могли лучше понять, как и когда их использовать.

Метод `__init__`.

Метод `__init__` — это так называемый «инициализатор». Он вызывается после создания объекта и служит для инициализации его атрибутов.



```
1 class Car:
2     def __init__(self, color):
3         self.color = color
4
5 car = Car("red") # создание объекта
6 print(car.color) # вывод: "red"
```

В этом примере метод `__init__` принимает два аргумента: `self` (это обычное название для ссылки на объект, с которым работает метод) и `color`. После создания объекта `car`, `__init__` вызывается автоматически с `car` в качестве `self` и строкой «red» в качестве `color`.

Метод `__del__`.

Метод `__del__` можно определить в классе, и он будет вызван автоматически, когда все ссылки на объект будут удалены или когда программа завершится.



```
1 class MyClass:
2     def __del__(self):
3         print("Удаление экземпляра")
4
5 obj = MyClass() # создание экземпляра
6 del obj # удаление экземпляра, вызывает __del__
```

В этом примере `__del__` просто выводит сообщение при удалении объекта. На практике `__del__` может быть использован для более сложных задач, таких как закрытие открытых файлов, освобождение сетевых ресурсов и т.д.

Практическая часть

В этой практической создадим класс Resource, который будет принимать значение название ресурса и тип ресурса, а также выводить их после удаление объекта.

1. Создаем класс Resource и добавляем магический метод `__init__`. С помощью магического метода `__init__` будем добавлять атрибуты объекта. В метод `__init__` добавляем аргументы `name`(название ресурса) и `resource_type`(тип ресурса). Теперь в самом методе надо присвоить значение из аргументов в сам объект, чтобы присвоить данные в объект используем параметр `self`.

```
1 class Resource:
2
3
4     def __init__(self, name: str, resource_type:str):
5         self.name = name
6         self.resource_type = resource_type
```

2. Создаем объекты класса Resource. Затем в круглы скобках вносим данные.

```
1 r1 = Resource("Соединение1", "подключение к базе данных")
2 r2 = Resource("Соединение2", "подключение к базе данных")
```

(чтобы обратиться к атрибуту объекта, напишите сам объект и через точку название атрибута объекта)

3. Добавляем магический метод `__del__`, который активируется при удалении объекта из памяти. Давайте его переопределим, и сделаем так, чтобы при удалении объекта он выводил данные из атрибутов.



```
1 def __del__(self):
2     print(f"Ресурс {self.name} типа {self.resource_type} удалён.")
```

4. Теперь запускаем файл с объектами класса. После запуска у вас сразу будет выводиться данные об объектах, это потому что объекты нигде не используются и их удаляет сборщик мусора(в python его называют GIL)
5. Давайте попробуем сами удалить объект класса с помощью оператора del. Создаем цикл for, который будет проводить 10 итераций, и на 5 будет удалять объект r2



```
1 for _ in range(1,11):
2     print(_)
3     if _ == 5:
4         del r2
```

После запуска программы в терминале выводится сначала пять чисел, а затем удаляется объект и выводит данные об объекте.

Задание

1. Создайте класс Node с атрибутами объекта data и next.
2. Создайте класс LinkedList.

Атрибутами класса:

- start - хранит начальный объект Node
- end - хранит конечный объект Node

Методы:

- len - возвращает количество связей
- search(data) - возвращает определенный объект из связанного списка по значению
- append(obj) - добавляет новую связь в конце связанного списка и изменяет атрибут класса end
- remove(index) - удаляет определенный объект по индексу, если удаляет первый объект по индексу, то изменяет атрибут класса start

Контрольные вопросы

1. Как работают метод __init__ и __del__.
2. Какая последняя строка вывелась после выполнения программы из практической части?

Практическое задание № 4

Параметры cls.
Магический метод `__new__`.

Теоретическая часть

Метод `__new__`.

Метод `__new__` — это настоящий конструктор в Python. Он вызывается до `__init__` и отвечает за создание (и возврат) нового объекта.

По умолчанию `__new__` создает экземпляр класса и возвращает его, но вы можете переопределить его для более сложного поведения. Например, вы можете использовать `__new__` для реализации паттернов проектирования, таких как singleton или factory.

```
1 class Singleton:
2     _instance = None # атрибут класса для хранения экземпляра
3
4     def __new__(cls, *args, **kwargs):
5         if cls._instance is None:
6             cls._instance = super().__new__(cls)
7         return cls._instance
8
9 s1 = Singleton()
10 s2 = Singleton()
11 print(s1 is s2) # True
```

В этом примере `__new__` проверяет, существует ли уже экземпляр Singleton, и если нет, вызывает `super().__new__(cls)` для создания нового. Если экземпляр уже существует, `__new__` просто возвращает его. Это гарантирует, что может существовать только один экземпляр Singleton.

Аргумент `cls`

Аргумент `cls` — это ссылка на класс, который вызывается. Он аналогичен аргументу `self`, который используется в методах экземпляра и указывает на конкретный объект.

Использование `cls` позволяет работать с классом в контексте методов класса, таких как `__new__` и другие классические методы.

Применение `cls`:

- Позволяет создавать новые экземпляры класса.
- Используется для доступа к атрибутам и методам самого класса.

Метод `super`

Метод `super` предоставляет доступ к методам родительского класса. Он используется для вызова методов, определенных в родительских классах, что особенно полезно в многократном наследовании. `super` помогает избежать явного указания имени родительского класса и обеспечивает более гибкое и безопасное взаимодействие между классами.

Основные моменты о `super`:

- **Вызывается для родительского класса:** Позволяет вызывать методы родительского класса без указания его имени.
- **Удобство в многократном наследовании:** Обеспечивает правильный порядок разрешения методов (Method Resolution Order, MRO).
- **Применение в `__init__`:** Часто используется для инициализации родительского класса в конструкторе.

Практическая часть

Создаем класс Singleton, в классе создаем атрибут класс `__instance`, который будет хранить объект класса.

```
1  class Singleton:
2      __instance = None
3
4
5      def __new__(cls, *args, **kwargs):
6          if cls.__instance is None:
7              cls.__instance = super().__new__(cls)
8          return cls.__instance
```

Переопределяем магический метод `__new__`. В методе `__new__` есть обязательный параметр `cls`, который ссылается на сам класс, а не на объект класса (`self` ссылается на объект класса). Также в параметрах, есть `*args` и `**kwargs`, они хранят все значение атрибута объекта в `set` и `dict`. Теперь в методе, пишем условие, которое будет проверят атрибут класса `__instance`, если в атрибуте объекта класса. При отсутствии в атрибуте класса, объекта класса будет создавать объект класса с помощью класса `super()` (класс `super()` - позволяет обратиться к родительскому классу и вызывать его методы, при создание класса они наследуются от объекта `object`, в котором находятся все магические методы). Вызываем метод `__new__`, чтобы создать объект класса.

Теперь создаем два объекта класса `Singleton`. Проверяем их идентичность это можно сделать с помощью оператора `is` или вывести объекты в терминал.

```
print(s1 is s2)
print(s2)
print(s1)
```

```
True
<__main__.Singleton object at 0x000001A690CC2A90>
<__main__.Singleton object at 0x000001A690CC2A90>
```

Задание

1. Создай класс, который мог создавать только пять объектов, если количество классов закончилось, то создавалась бы последнее созданный объект класса.
2. Создай класс Book:

Атрибуты объекта:

title

author

year

Который нельзя будет создать, одинаковыми атрибутами объектов title и author.

Контрольные вопросы

1. Чем отличаются метод `__new__` от `__init__`?
2. Для чего нужен класс `super()`?

Практическое задание № 5

Методы класса (classmethod)
Статические методы (staticmethod)

Теоретическая часть

Методы класса (classmethod) и статические методы (staticmethod) — это два специальных типа методов в Python, которые имеют различные применения в рамках объектно-ориентированного программирования (ООП).

1. `staticmethod`:

- a. **Описание:** Статический метод — это метод, который не получает неявного первого аргумента, связанного с объектом или классом. Он не имеет доступа к атрибутам экземпляра или класса.
- b. **Синтаксис:** Статический метод объявляется с использованием декоратора `@staticmethod`.
- c. **Пример использования:** Статические методы полезны, когда метод не нуждается в доступе ни к экземпляру, ни к классу, а предназначен для выполнения какой-то вспомогательной задачи.



```
1 class MyClass:
2     @staticmethod
3     def greet():
4         print("Hello from static method!")
5
6 MyClass.greet() # Вызов метода напрямую через класс
```

2. `classmethod`:

- **Описание:** Метод класса получает класс как неявный первый аргумент вместо экземпляра. Это позволяет работать с самим классом, а не с его конкретным объектом.

- **Синтаксис:** Методы класса объявляются с использованием декоратора `@classmethod`.
- **Пример использования:** Такие методы полезны, если нужно создать фабричный метод, который создаст экземпляры класса с помощью логики, зависящей от класса, или при необходимости работы с атрибутами класса.



```
1  class MyClass:
2      count = 0
3
4      def __init__(self):
5          MyClass.count += 1
6
7      @classmethod
8      def get_count(cls):
9          return cls.count
10
11  obj1 = MyClass()
12  obj2 = MyClass()
13  print(MyClass.get_count())  # Выведет: 2
14
```

Практическая часть

В этой практической создадим класс `CardCheck`, который будет проверять корректность вводимой информации о карте. Импортируем библиотеку `re`, а также из библиотеки `string` импортируем переменные `ascii_lowercase`, `digits`. В классе `CardCheck` создаем атрибут класса `CHARS_FOR_NAME`, в котором будут храниться переменные `ascii_lowercase` (используем метод `upper()`, чтобы все буквы были в верхнем регистре), `digits`.

```
1 class CardCheck:
2     CHARS_FOR_NAME = ascii_lowercase.upper() + digits
```

Создаем статический метод `check_card_number`, который будет проверять номер карты. Создаем шаблон регулярных выражений с помощью которого будем проверять номер карты.

```
1 @staticmethod
2 def check_card_number(number):
3     pattern = r'^\d{4}-\d{4}-\d{4}-\d{4}$'
4     return bool(re.match(pattern, number))
```

Создаем метод класса `check_name`, который будет проверять имя и фамилию владельца карты.

```
1  @classmethod
2  def check_name(cls, name):
3      words = name.split()
4      if len(words) != 2:
5          return False
6
7      first_name, last_name = words
8      return all(char in cls.CHARS_FOR_NAME for char in first_name) and \
9              all(char in cls.CHARS_FOR_NAME for char in last_name)
```

Задание

1. Напишите класс `TemperatureConverter`, который имеет:
 - a. Статический метод `celsius_to_fahrenheit` для перевода температуры из градусов Цельсия в Фаренгейты.
 - b. Метод класса `from_kelvin`, который будет создавать экземпляр, принимая температуру в Кельвинах и автоматически переводя её в Цельсия.
2. Создайте класс `Employee`, который будет представлять сотрудника и иметь следующие функции:
 1. Статический метод `is_valid_age`, который принимает возраст сотрудника и проверяет, находится ли возраст в допустимом диапазоне (например, от 18 до 65 лет).
 2. Метод класса `from_string`, который принимает строку с данными сотрудника в формате "Имя, Возраст, Должность" и создает экземпляр класса `Employee`, если возраст является допустимым. В противном случае метод должен выбросить исключение `ValueError`.
 3. Метод экземпляра `get_details`, который возвращает строку с информацией о сотруднике (например, "Имя: Иван, Возраст: 30, Должность: Менеджер").

Контрольные вопросы

1. В чем разница между `staticmethod` и `classmethod`?
2. Можно ли использовать `self` внутри метода, помеченного как `staticmethod`? Почему?

Практическое задание № 6

Режимы доступа public, private, protected.
Сеттеры и геттеры

Теоретическая часть

В объектно-ориентированном программировании (ООП) в Python используются различные режимы доступа для управления видимостью атрибутов и методов классов. Основные из них:

1. **Public (публичный)**: Атрибуты и методы, доступные из любого места в программе. По умолчанию все атрибуты в Python являются публичными.
2. **Private (приватный)**: Атрибуты и методы, доступные только внутри класса. В Python они определяются с помощью двойного подчеркивания перед именем (например, `__private_attr`). Это создает "имя mangling", что затрудняет доступ к ним из вне.
3. **Protected (защищенный)**: Атрибуты и методы, доступные в классе и его подклассах. Определяются с помощью одного подчеркивания перед именем (например, `_protected_attr`). Это лишь соглашение, но подразумевает, что доступ к ним следует ограничить.

Сеттеры и Геттеры

Сеттеры и геттеры — это методы, используемые для доступа и изменения атрибутов класса. Это полезно для инкапсуляции, позволяя контролировать доступ к данным.

- **Геттер**: Метод, который возвращает значение атрибута. Обычно называется `get_<имя_атрибута>()`.
- **Сеттер**: Метод, который устанавливает значение атрибута. Обычно называется `set_<имя_атрибута>()`.

Рассмотрим пример класса с использованием режимов доступа, сеттеров и геттеров:

```
1 class Person:
2     def __init__(self, name, age):
3         self._name = name # защищенный атрибут
4         self.__age = age  # приватный атрибут
5
6     # Геттер для имени
7     def get_name(self):
8         return self._name
9
10    # Сеттер для имени
11    def set_name(self, name):
12        self._name = name
13
14    # Геттер для возраста
15    def get_age(self):
16        return self.__age
17
18    # Сеттер для возраста с проверкой
19    def set_age(self, age):
20        if age >= 0:
21            self.__age = age
22        else:
23            raise ValueError("Возраст не может быть отрицательным")
24
25    # Пример использования
26    person = Person("Alice", 30)
27    print(person.get_name()) # Alice
28    print(person.get_age())  # 30
29
30    person.set_age(35)
31    print(person.get_age())  # 35
32
33    # person.__age = -10 # Это вызовет ошибку, так как __age приватен
34
```


Практическая часть

В этой практической мы создадим класс **Book**, который будет представлять книгу и использовать режимы доступа, сеттеры и геттеры.

```
1 class Book:
2     def __init__(self, title, author, pages):
3         self.title = title           # Публичный атрибут
4         self._author = author        # Защищенный атрибут
5         self.__pages = pages         # Приватный атрибут
6
```

- **title**: Публичный атрибут, доступный для чтения и записи из любого места.
- **_author**: Защищенный атрибут, который мы не рекомендуем изменять напрямую, но доступен в подклассах.
- **__pages**: Приватный атрибут, доступный только внутри класса.

Теперь добавим методы для доступа к защищенным и приватным атрибутам.

```
1 # Геттер для автора
2 def get_author(self):
3     return self._author
4
5 # Сеттер для автора
6 def set_author(self, author):
7     self._author = author
8
9 # Геттер для страниц
10 def get_pages(self):
11     return self.__pages
12
13 # Сеттер для страниц с проверкой
14 def set_pages(self, pages):
15     if pages > 0:
16         self.__pages = pages
17     else:
18         raise ValueError("Количество страниц должно быть положительным числом.")
```

Геттеры (`get_author`, `get_pages`) возвращают значения защищенных и приватных атрибутов. Сеттеры (`set_author`, `set_pages`) устанавливают значения, при этом `set_pages` проверяет, чтобы количество страниц было положительным.

Добавим метод для вывода информации о книге.

```
1 # Метод для отображения информации о книге
2 def display_info(self):
3     return f"Название: '{self.title}', Автор: {self._author}, Страниц: {self.__pages}"
4
```

Метод `display_info` возвращает строку с информацией о книге.

Задание

1. Создайте класс `Car`, который имеет следующие атрибуты:
 - Публичный атрибут `model`.
 - Защищенный атрибут `_year`.
 - Приватный атрибут `__mileage`.
- b. Реализуйте геттеры и сеттеры для всех атрибутов.
- c. Добавьте проверки в сеттеры, чтобы убедиться, что:
 - Год выпуска автомобиля не раньше 1886 и не позже текущего года.
 - Пробег не может быть отрицательным.

Контрольные вопросы

1. Почему важно использовать сеттеры и геттеры в ООП?
2. Приведите пример ситуации, когда использование защищенных атрибутов является более предпочтительным, чем использование публичных.

Практическое задание № 7

Свойства `property`. Декоратор `@property`

Теоретическая часть

Свойства (`property`) в Python — это механизм, позволяющий управлять доступом к атрибутам объекта класса. Вместо прямого доступа к атрибутам, вы можете использовать свойства для получения, установки или удаления их значений, при этом сохраняется интерфейс доступа через точечную нотацию, как для обычных атрибутов.

Основная задача свойств — это инкапсуляция данных, то есть скрывание внутренней реализации и предоставление интерфейса для работы с ними. Это позволяет контролировать процесс чтения и изменения атрибутов класса.

Свойства создаются с помощью функции `property()`, но чаще всего для этой цели используется декоратор `@property`. Декоратор делает код более читаемым и удобным для восприятия.

Пример без использования `@property`:

```
1 class Rectangle:
2     def __init__(self, width, height):
3         self._width = width
4         self._height = height
5
6     def get_width(self):
7         return self._width
8
9     def set_width(self, value):
10        if value > 0:
11            self._width = value
12        else:
13            raise ValueError("Ширина должна быть положительной!")
14
15    width = property(get_width, set_width)
```

Пример с использованием декоратора `@property`:

```
1 class Rectangle:
2     def __init__(self, width, height):
3         self._width = width
4         self._height = height
5
6     @property
7     def width(self):
8         return self._width
9
10    @width.setter
11    def width(self, value):
12        if value > 0:
13            self._width = value
14        else:
15            raise ValueError("Ширина должна быть положительной!")
16
```

В примере выше:

- Декоратор `@property` позволяет определить метод для чтения значения атрибута.
- Декоратор `@<имя_свойства>.setter` создаёт метод для установки значения.

Преимущества использования `@property`:

1. **Инкапсуляция** — вы можете скрывать реализацию атрибута и управлять его доступом через методы, не меняя интерфейс.
2. **Чистота кода** — доступ к свойствам выглядит как к обычным атрибутам, что улучшает читаемость.
3. **Контроль над изменениями** — можно добавить валидацию и обработку данных при изменении атрибутов.

Практическая часть

Создадим класс BankAccount с атрибута объекта balance для которого создадим getter и setter с помощью декоратора @property.



```
1 class BankAccount:
2     def __init__(self, initial_balance=0):
3         self._balance = initial_balance
```

Для начало создаем свойство getter, чтобы от этого свойства получать данные от атрибута объекта balance.(обязательно используем декоратор property)



```
1 @property
2 def balance(self):
3     return self._balance
```

После создание свойство getter можно создавать свойство setter для balance. Чтобы создать setter для balance используем название свойства getter как декоратор для setter.

```
1  @balance.setter
2  def balance(self, amount):
3      if amount < 0:
4          raise ValueError("Баланс не может быть отрицательным.")
5      self._balance = amount
6
```

В свойства setter проверяем входные данные на отрицательное число. Теперь когда будем создавать объект класса, мы не сможем дать атрибуту balance отрицательное число. Также добавим в класс методы deposit и withdraw.

```
1  def deposit(self, amount):
2      if amount <= 0:
3          raise ValueError("Сумма депозита должна быть положительной.")
4      self.balance += amount
5
6  def withdraw(self, amount):
7      if amount > self.balance:
8          raise ValueError("Недостаточно средств.")
9      self.balance -= amount
```

Задание

1. Создайте класс **Product** с атрибутами:
 - i. **name** (название),
 - ii. **price** (цена),
 - iii. **discount** (скидка в процентах).
 - b. Реализ:
 - i. Свойство **price_with_discount**, возвращающее цену с учетом скидки.
 - ii. Методы установки цены и скидки с валидацией:
 1. Цена ≥ 0 .
 2. Скидка от 0% до 100%.
2. Создайте класс **Employee** с атрибутами:
 - i. **name** (имя),
 - ii. **salary** (зарплата),
 - iii. **age** (возраст).
 - b. Реализ:
 - c. Свойства для имени и зарплаты (зарплата $\geq 30\,000$).
 - d. Свойство для возраста с возможностью удаления (возраст станет None после удаления).
 - e. Метод **apply_raise()**, увеличивающий зарплату на заданный

Контрольные вопросы

1. Чем декоратор `@property` отличается от обычных методов?
2. Какие преимущества даёт использование свойств по сравнению с прямым доступом к атрибутам?

Практическое задание № 8

Магические методы `__setattr__`,
`__getattr__`, `__getattribute__` и `__delattr__`

Теоретическая часть

Методы

Методы `__setattr__`, `__getattribute__`, `__getattr__` и `__delattr__` относятся к управлению атрибутами объектов и позволяют контролировать, как атрибуты устанавливаются, извлекаются и удаляются.

1. `__setattr__(self, name, value)`

- **Описание** : Этот метод вызывается, когда мы пытаемся установить значение атрибута объекта. Он принимает два аргумента: `name` — имя атрибута, и `value` — значение, которое мы хотим установить.
- **Примечание** : Чтобы избежать рекурсивного вызова самого себя при установке атрибутов, лучше использовать `super().__setattr__(name, value)`.
- **Пример** :



```
1 class Person:
2     def __init__(self, name):
3         self.name = name
4
5     def __setattr__(self, name, value):
6         print(f"Устанавливаем атрибут '{name}' со значением '{value}'")
7         super().__setattr__(name, value)
8
9 # Пример работы
10 p = Person("Alice")
11 p.age = 30 # Устанавливает атрибут age
12
```

2. `__getattribute__(self, name)`

- **Описание** : Этот метод вызывается при попытке доступа к любому атрибуту объекта. Он принимает один аргумент — имя атрибута.

- **Примечание** : Этот метод может вызывать рекурсивные вызовы, если вы не будете использовать `super()`. Он подходит для перехвата всех попыток доступа к атрибутам.
- **Пример** :



```
1 class Person:
2     def __init__(self, name):
3         self.name = name
4
5     def __getattr__(self, name):
6         print(f"Получаем атрибут '{name}'")
7         return super().__getattr__(name)
8
9 # Пример работы
10 p = Person("Bob")
11 print(p.name) # Получает атрибут name
12
```

3. `__getattr__(self, name)`

- **Описание** : Этот метод вызывается только в том случае, если стандартный механизм получения атрибутов (т.е. `__getattribute__`) не находит атрибут. Он может использоваться для создания «умных» атрибутов или для обработки отсутствующих атрибутов.
- **Примечание** : `__getattr__` полезен для реализации атрибутов с умолчаниями или вычисляемых атрибутов.
- **Пример**:



```

1 class Person:
2     def __init__(self, name):
3         self.name = name
4
5     def __getattr__(self, name):
6         print(f"Атрибут '{name}' не найден, возвращаем значение по умолчанию.")
7         return "Неизвестно"
8
9 # Пример работы
10 p = Person("Charlie")
11 print(p.age) # Попытка получить отсутствующий атрибут age
12

```

4. `__delattr__(self, name)`

- **Описание** : Этот метод вызывается при попытке удалить атрибут объекта. Он принимает один аргумент — имя атрибута, который нужно удалить.
- **Примечание** : Как и в случае с `__setattr__`, рекомендуется использовать `super().__delattr__(name)` для выполнения стандартного удаления.
- **Пример** :



```

1 class Person:
2     def __init__(self, name):
3         self.name = name
4
5     def __delattr__(self, name):
6         print(f"Удаляем атрибут '{name}'")
7         super().__delattr__(name)
8
9 # Пример работы
10 p = Person("David")
11 p.age = 25 # Устанавливаем атрибут age
12 del p.age # Удаляем атрибут age

```

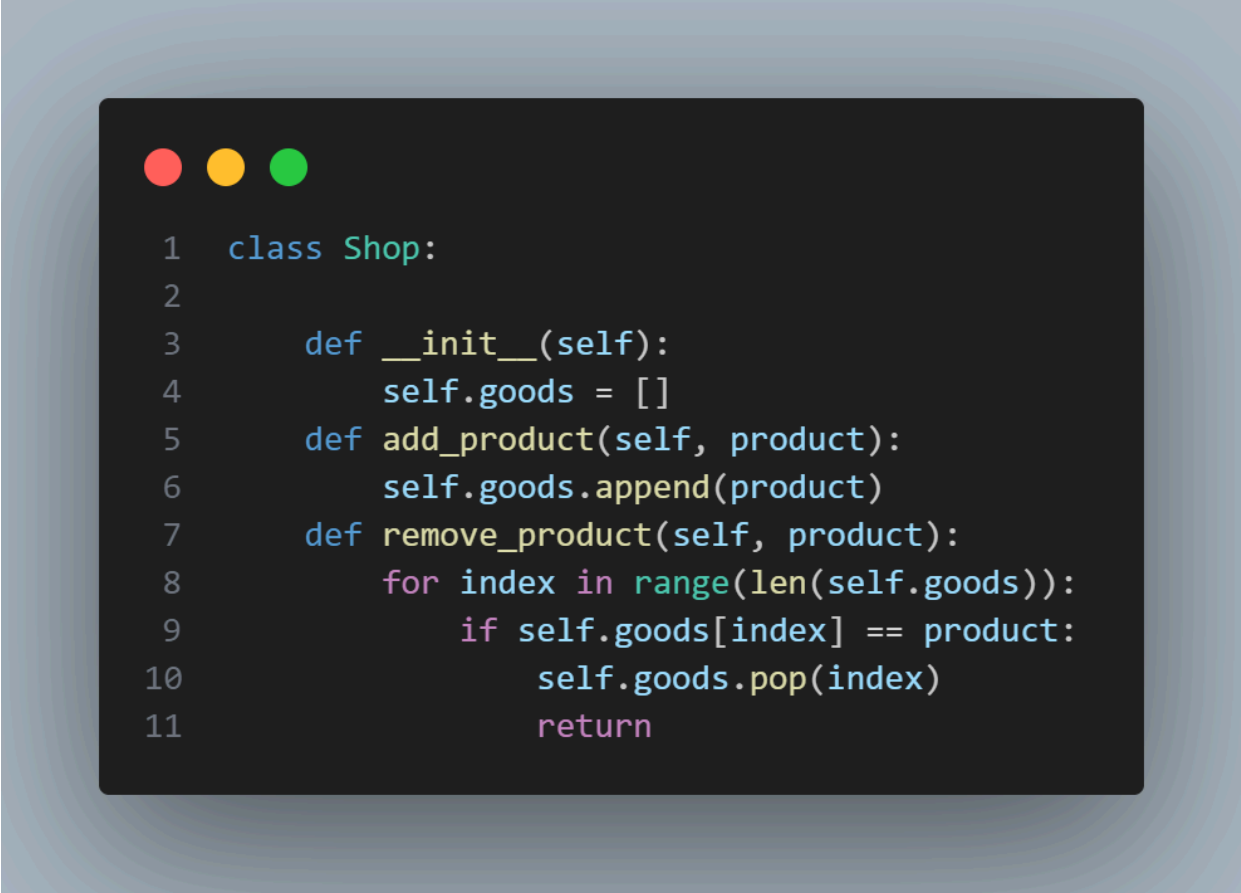
Практическая часть

Реализуем класс Shop, который будет хранить список продуктов, а также добавлять и удалять продукты.

Также создаем класс Product с атрибутам объекта:

- id - уникальный идентификационный номер товара (который генерируется автоматически)
- name - название продукта (строка)
- weight - вес товара (целое или вещественное положительное число)
- price - цена(целое или вещественное положительное число)

Создаем класс Shop с методами add_product и remove_product



```
1 class Shop:
2
3     def __init__(self):
4         self.goods = []
5     def add_product(self, product):
6         self.goods.append(product)
7     def remove_product(self, product):
8         for index in range(len(self.goods)):
9             if self.goods[index] == product:
10                 self.goods.pop(index)
11         return
```

Теперь реализуем класс Product. Для начало создадим приватный атрибут id, который будет хранить уникальный идентификационный номер товара. Чтобы реализовать такой функционал надо переопределять метод __new__.



```
1 class Product:
2     __id = 0
3     def __new__(cls, *args, **kwargs):
4         cls.__id+=1
5         obj = super().__new__(cls)
6         obj.id = cls.__id
7         return obj
```

В просто прибавляем к приватному атрибуту по единице и добавляем его к объекту, затем создаем объект.

Создаем объекты атрибутов.



```
1 def __init__(self, name:str, weight:float, price:float):
2     self.name = name
3     self.weight = weight
4     self.price = price
```

Переопределять `__setattr__`, чтобы создать валидацию для атрибутов объекта. Создаем словарь с названиями атрибутов и значениями в виде типов данных. С помощью словаря проверяем присваиваемые значения.



```
1 def __setattr__(self, name: str, value) -> None:
2     type_dict = {
3         "id": int,
4         "name": str,
5         "weight": (int,float),
6         "price": (int,float),
7     }
8
9     if not isinstance(value, type_dict[name]):
10         raise TypeError("Неверный тип присваиваемых данных.")
```

Еще добавляем проверку на отрицательные числа у атрибутов weight и price.



```
1 if name == 'weight' and not value > 0 :
2     raise TypeError("Неверный тип присваиваемых данных.")
3 if name == 'price' and not value > 0 :
4     raise TypeError("Неверный тип присваиваемых данных.")
5 super().__setattr__(name, value)
```

Переопределять метод `__delattr__`, чтобы нельзя было удалить атрибут `id`.

```
1 def __delattr__(self, name: str) -> None:
2     if name == 'id':
3         raise AttributeError("Атрибут id удалять запрещено.")
4     super().__delattr__(name)
```

Задание

1. Измените класс Shop, чтобы он хранил список товаров в виде словаря, также изменить методы.
2. **Класс Library:**
 - **Атрибуты:**
 - name (строка): название библиотеки.
 - books (список): список книг (каждая книга — строка с названием).
 - max_books (целое число): максимальное количество книг, которое может храниться в библиотеке.
 - **Магические методы:**
 - **__setattr__(self, name, value):**
 - Ограничивает количество книг в библиотеке (при добавлении книги).
 - Запрещает установку max_books после инициализации.
 - **__getattr__(self, name):**
 - Логирует доступ к атрибутам, выводя сообщение о том, к какому атрибуту происходит доступ.
 - **__getattr__(self, name):**
 - Возвращает сообщение, если запрашиваемый атрибут не найден.
 - **__delattr__(self, name):**
 - Запрещает удаление атрибута name библиотеки.
 - Выводит сообщение о том, какой атрибут удаляется.
 - **Методы:**
 - **add_book(self, book):** добавляет книгу в библиотеку, если количество книг меньше max_books.
 - **remove_book(self, book):** удаляет книгу из библиотеки, если она существует.
 - **list_books(self):** возвращает список всех книг в библиотеке.

Контрольные вопросы

1. Какие практические применения есть у магического метода __setattr__?
2. В чем разница между __getattr__ и __getattr__?

Практическое задание № 9

Магический метод `__call__`

Теоретическая часть

Магический метод `__call__` в Python — это специальный метод, который позволяет экземпляру класса вести себя как функцию, а не как обычный объект. Это означает, что мы можем «вызывать» экземпляры класса так, как если бы они были функциями, и не вносить какие-либо изменения в код, который использует эти экземпляры.

Базовое использование метода `__call__`

В самом простом случае, `__call__` может быть использован без аргументов для определения объекта, который ведет себя как функция:

A screenshot of a code editor with a dark background and light green accents. The code is written in Python and defines a class named 'CallableClass' with a method '__call__'. An instance of the class is created and called, resulting in a printed message.

```
test/call.py

class CallableClass:
    def __call__(self):
        print("Метод __call__ был вызван")

cc = CallableClass()
cc() # Вывод: "Метод __call__ был вызван"<Up>
```

Здесь мы видим, что экземпляр класса `CallableClass` может быть «вызван», как если бы он был функцией. В результате вызова будет выполнен код, определенный в методе `__call__`.

Метод `call` с аргументами

Метод `__call__` также может принимать аргументы, которые передаются при «вызове» экземпляра класса:

```
test/call.py

class CallableClass:
    def __call__(self, value):
        print(f"Метод __call__ был вызван со значением: {value}")

cc = CallableClass()
cc(42) # Вывод: "Метод __call__ был вызван со значением: 42"
```

Использование call для создания функциональных объектов

Использование `__call__` позволяет создавать классы, объекты которых могут имитировать функциональное поведение. Это особенно полезно, если вам нужно сохранить состояние между вызовами функции, поскольку все переменные экземпляра сохраняются между вызовами:

```
test/call.py

class Counter:
    def __init__(self):
        self.count = 0

    def __call__(self):
        self.count += 1
        print(f"Текущее значение счетчика: {self.count}")

c = Counter()
c() # Вывод: "Текущее значение счетчика: 1"
c() # Вывод: "Текущее значение счетчика: 2"
```


Здесь класс Counter имитирует поведение функции, которая увеличивает свое внутреннее значение каждый раз, когда она вызывается.

Магический метод `__call__` в декораторах

Метод `__call__` часто используется в реализации декораторов, что дает большую гибкость по сравнению с обычными функциями. Декоратор — это функция, которая принимает другую функцию в качестве аргумента, и может выполнить дополнительный код до или после этой функции.

test/call.py

```
class Logger:
    def __init__(self, function):
        self.function = function

    def __call__(self, *args, **kwargs):
        print(f"Начинаем выполнение {self.function.__name__}")
        result = self.function(*args, **kwargs)
        print(f"Закончили выполнение {self.function.__name__}")
        return result

@Logger
def say_hello(name):
    print(f"Привет, {name}!")

say_hello("Алиса")

# Начинаем выполнение say_hello
# Привет, Алиса!
# Закончили выполнение say_hello
```

В этом примере **Logger** является классом декоратора, который «оборачивает» функцию и выводит сообщения перед и после ее выполнения.

Практическая часть

В этой практической сделаем класс **Handler**, который будет использоваться как декоратор для проверки http запросов.

Для начала создадим класс `Handler`, и с помощью `__init__` определим аргументы для декоратора. Декоратор будет принимать только один аргумент `methods`.

A screenshot of a code editor with a dark background and light green accents. At the top left, there are three colored circles (red, yellow, green) and the file path 'test/call.py'. The code defines a class 'Handler' with an '__init__' method that takes 'self' and 'methods' as arguments, with 'methods' defaulting to 'GET'.

```
test/call.py
```

```
class Handler:
```

```
    def __init__(self, methods=("GET", )):
        self.methods = methods
```

Также для аргумента `methods` сделали изначальное значение `GET` в `set`, чтобы значение не повторялись.

Теперь создаем функционал для метода `__call__`. В аргументах метода после `self`, добавляем `func` – это функция которую мы обернули в декоратор. Также добавим `*args` и `**kwargs` для дополнительных аргументов.

В самом методе создаем словарь с ключом (название метода, который его исполняет) и значением (метод, который его исполняет)



test/call.py

```
def __call__(self, func, *args, **kwargs):  
    type_methods = {  
        "GET": self.get,  
        "POST": self.post,  
    }
```

Затем создаем функцию wrapper(в методе __call__) , которая будет прослойкой для обработки входящих данных. В параметрах __call__ добавляем атрибуты, которые находятся в оборачиваемой функции.

Проверяем какой запрос к нам пришел. Для этого пишем условие, которое будет проверять данные request с типами запросов указанные в декораторе.

test/call.py

```
def wrapper(request, *args, **kwargs):  
    if request["methods"] not in self.methods:  
        raise f'Данный страница не принимает тип запросов {request['methods']}'
```

После всех проверок, пишем возвращаем метод, который будет взаимодействовать с определенным запросом(эти методы мы писали в type_dict).



```
test/call.py
return type_methods[request["methods"]](func, request, *args, **kwargs)
```

Методы из `type_dict` (это методы класса, а не функция в методе `__call__`):



```
test/call.py

def get(self, func, request, *args, **kwargs):
    return func(request, *args, **kwargs)

def post(self, func, request, *args, **kwargs):
    return func(request, *args, **kwargs)
```

После всех проверок магический метод `__call__` возвращает объект функции `wrapper`.

Теперь для проверки создаем функцию `get_page` с параметром `request`. В `request` будет некий запрос от клиента, который мы будем обрабатывать.

```
test/call.py

@Handler(methods=("POST",))
def get_page(request):
    return "Привет мир"

print(
    get_page(
        {
            "methods": "POST",
        }
    )
)
```

Задание

1. Создайте класс **Power**, который принимает степень **n** при инициализации, а при вызове экземпляра класса возводит переданный аргумент в степень **n**
2. Создайте класс-декоратор **Repeat**, который будет повторять выполнение функции определённое количество раз, которое передаётся при инициализации.

Контрольные вопросы

1. Как можно вызвать метод `__call__`, не обращаясь к нему напрямую?
2. Какие преимущества даёт использование метода `__call__` по сравнению с обычными функциями?

Практическое задание № 10

Магические методы `__str__`, `__repr__`, `__len__`, `__abs__`

Теоретическая часть

Рассмотрим подробно магические методы `__str__`, `__repr__`, `__len__` и `__abs__`.

Магический метод `__str__`

Метод `__str__` используется для определения "читаемого" представления объекта, которое возвращает строку, предназначенную для пользователя. Если метод определён, то `print(object)` или `str(object)` вызовут `__str__`.



test/methods.py

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"{self.name}, age: {self.age}"

person = Person("Alice", 30)
print(person)  # Вывод: Alice, age: 30
```

Магический метод `__repr__`

Метод `__repr__` определяет строковое представление объекта, ориентированное на разработчиков. Часто оно используется для отладки и обычно содержит полную информацию, чтобы при копировании и вставке выводилась точная копия объекта. В идеале `__repr__` должно возвращать строку, которую можно использовать для создания такого же объекта.

test/methods.py

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __repr__(self):
        return f"Person(name='{self.name}', age={self.age})"

person = Person("Alice", 30)
print(repr(person)) # Вывод: Person(name='Alice', age=30)
```

Различия между `__str__` и `__repr__`

Основное различие между методами `__str__` и `__repr__` заключается в их предназначении и том, для кого они ориентированы.

1. `__str__` предназначен для вывода, понятного конечному пользователю. Это более "человекоориентированное" представление, которое должно быть читабельным и информативным. `__str__` чаще всего используется для отображения объекта через `print` или `str()`.
2. `__repr__` ориентирован на разработчиков и используется для отображения объекта таким образом, чтобы можно было его "воссоздать". Обычно вывод `__repr__` содержит более детализированное и технически точное представление объекта, чтобы это выражение можно было использовать для создания идентичного объекта, если возможно. Это помогает при отладке, так как показывает структуру и данные объекта.

Если в классе реализован только `__repr__` и отсутствует `__str__`, то при вызове `print(object)` Python будет использовать `__repr__` в качестве запасного варианта

```
test/methods.py
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __str__(self):
        return f"{self.name}, age: {self.age}"

    def __repr__(self):
        return f"Person(name='{self.name}', age={self.age})"

person = Person("Alice", 30)
print(str(person)) # Вывод для пользователя: "Alice, age: 30"
print(repr(person)) # Вывод для разработчика: "Person(name='Alice', age=30)"
```

Магический метод `__len__`

Метод `__len__` используется для определения длины объекта, то есть позволяет объекту отвечать на вызов `len(object)`. Этот метод полезен для коллекций, где можно определить количество элементов, но также его можно использовать для любого объекта, где имеет смысл возвращать длину.



test/methods.py

```
class Group:
    def __init__(self, members):
        self.members = members

    def __len__(self):
        return len(self.members)

group = Group(["Alice", "Bob", "Charlie"])
print(len(group)) # Вывод: 3
```

Магический метод `__abs__`

Метод `__abs__` определяет, что должно возвращаться при вызове `abs(object)`. Обычно его используют для чисел или объектов, где можно определить абсолютное значение.



test/methods.py

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __abs__(self):
        return (self.x**2 + self.y**2) ** 0.5

point = Point(3, 4)
print(abs(point)) # Вывод: 5.0
```

Практическая часть

В этой практической напиши класса Model с методом query. Метод query будет принимать имя поля и значение для того поля, затем сохранять его в классе. Также в классе будет использовать магический метод `__str__` и `__len__`.

Магический метод `__str__` будет выводить все ключи и значения сохраненные в класса.

Метод `__len__` будет выводить количество полей.

Создаем класс Model с методом query.



```
test/methods.py
```

```
class Model:
    def query(self, *args, **kwargs):
        for key, item in kwargs.items():
            self.__dict__[key] = item
```

В методе query используем `*args` и `**kwargs`. В `*args` будет храниться аргументы по порядку в виде `set`, а в `**kwargs` будет храниться аргументы с ключем и значениям в виде `dict`(можно использовать любое название вместо `args` и `kwargs`, но обязательно использовать звездочки).

Переопределяем метод `__str__`, чтобы он выводил все параметры объекта.



test/methods.py

```
def __str__(self):  
    result = []  
    for key, item in self.__dict__.items():  
        result.append(f"{key} = {item}")  
  
    return "Model: " + ",".join(result)
```

Теперь при выводе объекта в терминал будет показываться все его атрибуты, а не блок памяти который он занимает.

Переопределяем метод `__len__`.



test/methods.py

```
def __len__(self):  
    return len(self.__dict__)
```

Теперь при использовании функции `len()` для объекта класса `Model` будет выводиться количество полей в объекте.

Задание

1. Напишите класс **ShoppingCart**, который хранит список товаров. Добавьте метод `__len__`, который возвращает количество товаров в корзине.
2. Создайте класс **Book**, который содержит свойства `title`, `author` и `pages` (количество страниц). Определите магические методы `__str__`, `__repr__` и `__len__`.
 - `__len__` - возвращает количество страниц
 - `__str__` - выводит название класса и все поля со значениями
 - `__repr__` - выводит название класса, а также все названия полей и предполагаемые типы данных

Контрольные вопросы

1. В чем разница между методами `__str__` и `__repr__`?
2. Что произойдет, если вызвать `print(object)` для объекта, у которого не определен метод `__str__`?
3. Какую задачу решает метод `__abs__`, и в каких случаях его реализация может быть полезной?

Практическое задание № 11

Методы `__add__`, `__sub__`, `__mul__`, `__truediv__`

Теоретическая часть

Магические методы (или методы перегрузки операторов) в Python позволяют определить поведение операторов (+, -, *, / и других) при работе с пользовательскими объектами. Эти методы имеют определенные названия, начинающиеся и заканчивающиеся двумя нижними подчеркиваниями. Основное преимущество магических методов в том, что они позволяют сделать классы более интуитивно понятными и удобными в использовании.

Четыре магических метода, которые мы рассмотрим:

- `__add__(self, other)` — используется для перегрузки оператора сложения +.
- `__sub__(self, other)` — используется для перегрузки оператора вычитания -.
- `__mul__(self, other)` — используется для перегрузки оператора умножения *.
- `__truediv__(self, other)` — используется для перегрузки оператора деления /.

Когда мы используем эти операторы с объектами пользовательского класса, Python автоматически вызывает соответствующий магический метод, если он определен.

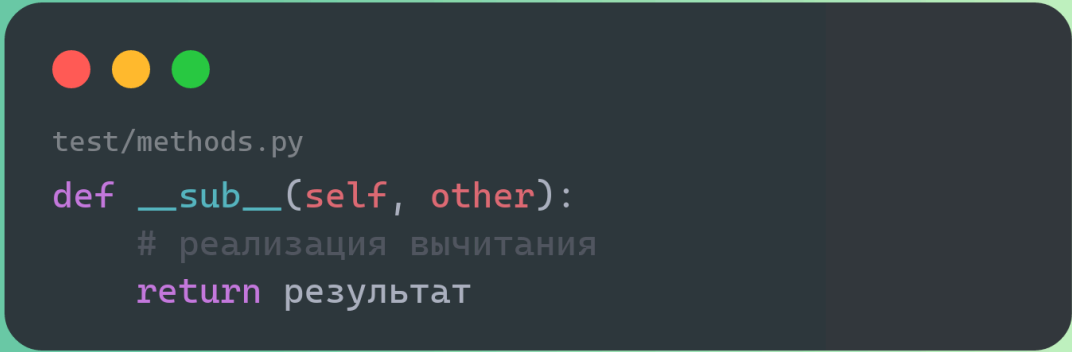
`__add__(self, other)`

Метод `__add__` отвечает за сложение объектов. Он принимает два аргумента — `self` (ссылка на текущий экземпляр класса) и `other` (второй операнд). Метод возвращает результат сложения.



`__sub__(self, other)`

Метод `__sub__` отвечает за вычитание объектов. Аналогично `__add__`, он принимает два аргумента и возвращает разность.

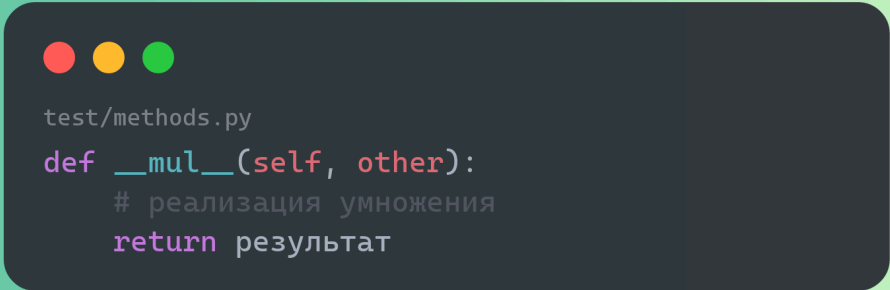
A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text inside the terminal is as follows:

```
test/methods.py
def __sub__(self, other):
    # реализация вычитания
    return результат
```

```
test/methods.py
def __sub__(self, other):
    # реализация вычитания
    return результат
```

`__mul__(self, other)`

Метод `__mul__` используется для умножения объектов.

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The text inside the terminal is as follows:

```
test/methods.py
def __mul__(self, other):
    # реализация умножения
    return результат
```

```
test/methods.py
def __mul__(self, other):
    # реализация умножения
    return результат
```

`__truediv__(self, other)`

Метод `__truediv__` используется для деления объектов.



test/methods.py

```
def __truediv__(self, other):  
    # реализация деления  
    return результат
```


Практическая часть

В этой практической создадим классы Library и Book. Library переопределим метод `__add__` и `__sub__`, чтобы добавлять объекты класса Book с помощью арифметических операторов.

Создаем класс Book с атрибутами объекта: title, year, author. Затем создаем класс Library с атрибутом класса data в котором будет храниться пустой словарь.

Переопределяем метод `__add__`.

```
test/methods.py
```

```
def __add__(self, other):  
    if not self.data.get(other.author):  
        self.data[other.author] = [other]  
        return self  
    self.data[other.author].append(other)  
  
    return self
```

В данном методе есть обязательный параметр other, он отвечает за объект который мы плюсуем (это объект который находится справа, грубо говоря). Прописываем логику, которая проверяет на существование данной книги в словаре, если данного объекта не существует, то добавляем его в словарь с ключом автора, а значением является объект со списком (в данном примере). В другом случае просто добавляем его в список. Во всех случаях возвращаем сам объект.

Переопределяем метод `__sub__`



test/methods.py

```
def __sub__(self, other):  
    if other.author not in self.data.keys():  
        return  
  
    if not len(self.data[other.author]):  
        del self.data[other.author]  
        return self  
    self.data.pop(other.author)  
  
    return self
```

Теперь добавляем логику удаление книги из библиотеки. Тут два условия, которые определяют существует ли данный объект в словаре, а второй проверяет на последний объект в списке.

Создаем объект класса Library, а также несколько объектов класса Book.



test/methods.py

```
lib = Library()  
  
book = Book("Горе от ума", "Александр Грибоедов", 1825)  
book2 = Book("Портрет Дориана Грея", "Оскар Уайльд", 2020)  
  
lib + book  
lib + book2  
  
lib - book  
print(lib.data)
```

Задание

1. Создайте класса **List**, который будет наследовать от класса `list`. И переопределите методы:
 - a. `__add__` – добавляет объект в список
 - b. `__sub__` – удалит объект из списка по значению
 - c. `__mul__` – делит список(делит список в большинстве)
2. Создайте односвязанный список, который будет использовать арифметические операторы вместо методов добавление и вычитания.

Контрольные вопросы

1. Какой тип данных передается в `other` при вызове `__add__` или `__sub__`?
2. Почему перегрузка операторов полезна при разработке на Python?

Практическое задание № 12

Магические методы `__eq__`, `__ne__`, `__lt__`, `__gt__`, `__le__`, `__ge__`

Теоретическая часть

Методы для операций сравнения (`__eq__`, `__ne__`, `__lt__`, `__gt__`, `__le__`, `__ge__`) позволяют определить, как объекты класса будут сравниваться между собой.

1. **`__eq__(self, other)`**

Определяет поведение оператора `==`. Возвращает `True`, если объекты равны, и `False`, если нет.

Пример: `if obj1 == obj2`

2. **`__ne__(self, other)`**

Определяет поведение оператора `!=`. Возвращает `True`, если объекты не равны.

Пример: `if obj1 != obj2`

3. **`__lt__(self, other)`**

Определяет поведение оператора `<`. Возвращает `True`, если `self` меньше `other`.

Пример: `if obj1 < obj2`

4. **`__gt__(self, other)`**

Определяет поведение оператора `>`. Возвращает `True`, если `self` больше `other`.

Пример: `if obj1 > obj2`

5. **`__le__(self, other)`**

Определяет поведение оператора `<=`. Возвращает `True`, если `self` меньше или равно `other`.

Пример: `if obj1 <= obj2`

6. **`__ge__(self, other)`**

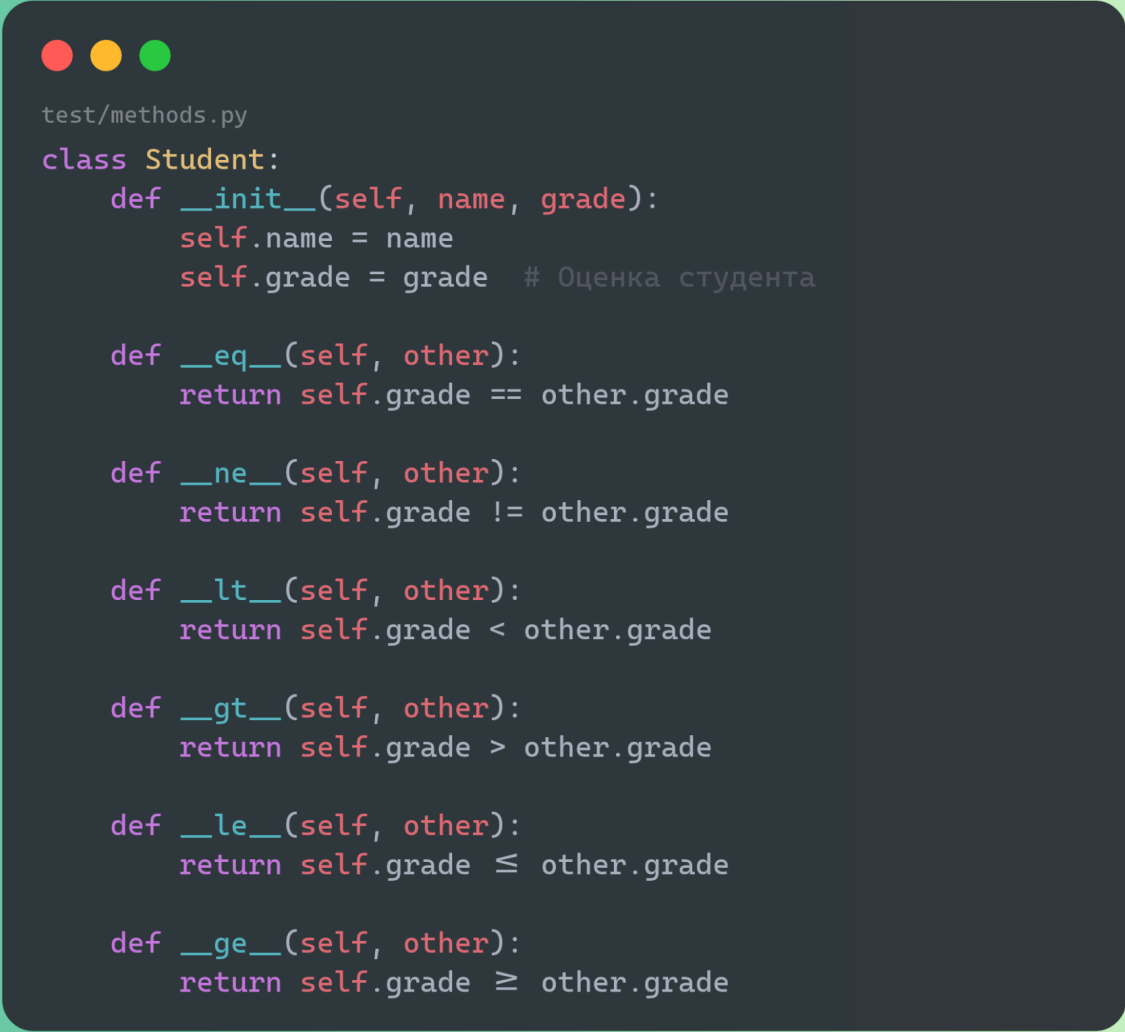
Определяет поведение оператора `>=`. Возвращает `True`, если `self` больше или равно `other`.

Пример: `if obj1 >= obj2`

Эти методы принимают два аргумента: `self` (текущий объект) и `other` (объект для сравнения). Если `other` не относится к ожидаемому типу, обычно возвращается `NotImplemented`.

Практическая часть

В этой практической реализуем класс `Student` с атрибутами объекта `name` и `grade`. В классе преопределим методы сравнений.



```
test/methods.py
class Student:
    def __init__(self, name, grade):
        self.name = name
        self.grade = grade # Оценка студента

    def __eq__(self, other):
        return self.grade == other.grade

    def __ne__(self, other):
        return self.grade != other.grade

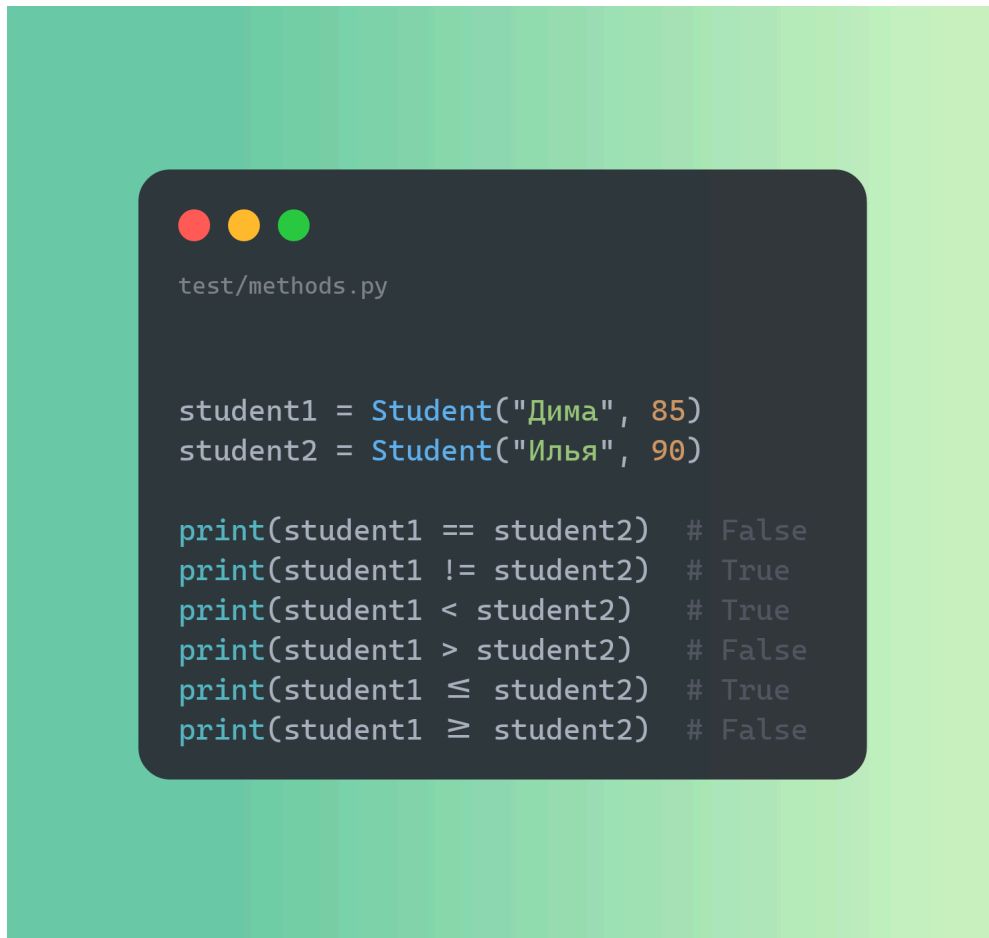
    def __lt__(self, other):
        return self.grade < other.grade

    def __gt__(self, other):
        return self.grade > other.grade

    def __le__(self, other):
        return self.grade ≤ other.grade

    def __ge__(self, other):
        return self.grade ≥ other.grade
```

В методах сравнения добавили логику сравнение с объектами класса `Student` используя атрибут класса `grade`. Теперь для примера создадим два объекта класса `Student`, и с их помощью проверим данные методы.



```
test/methods.py

student1 = Student("Дима", 85)
student2 = Student("Илья", 90)

print(student1 == student2) # False
print(student1 != student2) # True
print(student1 < student2)  # True
print(student1 > student2)  # False
print(student1 ≤ student2)  # True
print(student1 ≥ student2)  # False
```

Задание

1. Создайте класс Book с атрибутами объекта title и pages. Реализуйте все методы сравнений используя атрибут объекта pages.

Контрольные вопросы

1. Чем магические методы `__eq__` и `__ne__` отличаются от `__lt__` и `__gt__`?
2. Какое значение должны возвращать магические методы сравнения?

Практическое задание № 13

Магические методы `__eq__` и `__hash__`

Теоретическая часть

В объектно-ориентированном программировании Python методы `__eq__` и `__hash__` играют важную роль при сравнении объектов и работе с хэш-таблицами, такими как словари (dict) и множества (set).

Метод `__eq__`

Метод `__eq__(self, other)` определяет поведение оператора равенства (`==`). Он используется для проверки равенства двух объектов. По умолчанию, этот метод сравнивает ссылки на объекты, то есть проверяет, ссылаются ли оба оператора на один и тот же объект в памяти.

Пример:



```
methods.py
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __eq__(self, other):
        if isinstance(other, Point):
            return self.x == other.x and self.y == other.y
        return False

p1 = Point(1, 2)
p2 = Point(1, 2)
print(p1 == p2) # True, так как __eq__ сравнивает значения x и y
```

Метод `__hash__`

Метод `__hash__(self)` возвращает хэш-значение объекта, используемое для его идентификации в хэш-таблицах. Если вы переопределяете `__eq__`, то переопределение `__hash__` также необходимо для правильной работы с объектами в множествах или словарях.

Правило: Если два объекта равны по `__eq__`, их хэши должны быть одинаковыми. Но обратное не обязательно: два объекта с одинаковыми хэшами могут быть не равны (hash collision).

Пример:

```
methods.py
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __eq__(self, other):
        if isinstance(other, Point):
            return self.x == other.x and self.y == other.y
        return False

    def __hash__(self):
        return hash((self.x, self.y))

p1 = Point(1, 2)
p2 = Point(1, 2)
print(hash(p1) == hash(p2)) # True, так как __hash__ использует те же атрибуты
```

- Объекты с переопределённым `__eq__` и без корректного `__hash__` нельзя использовать в качестве ключей в словарях или элементов множеств.
- Если объект неизменяемый, то `__hash__` можно реализовать на основе его атрибутов. Для изменяемых объектов чаще всего `__hash__` возвращает `None`.

Практическая часть

В этой практической реализуем класс ShopItem с атрибутами объекта name, weight, price.

Используя атрибуты объекта, будем создавать хэш в методе __hash__ для объекта класса ShopItem, а также переопределим магический метод __eq__, чтобы он сравнивал его по хэшу функции.

Создаем класс ShopItem:



methods.py

```
class ShopItem:
    def __init__(self, name, weight, price) -> None:
        self.name = name
        self.weight = weight
        self.price = price
```

Переопределяем магический метод __hash__.

Чтобы создавать хэш используем функцию hash() или создаем свою функцию хеширования.

Для использования hash() нужно передать один аргумент, который будет неизменяемые. Из-за передаем неизменяемы список (tuple) с нашими аргументами.



methods.py

```
def __hash__(self) -> int:  
    return hash((self.name, self.weight, self.price))
```

Теперь переопределяем магический метод `__eq__` с использованием магический метод `__hash__`.



methods.py

```
def __eq__(self, value: ShopItem) -> bool:  
    if isinstance(value, ShopItem):  
        return self.__hash__() == value.__hash__()  
    return False
```

В методе `__eq__` пишем условие, которое проверяет объект на принадлежность к классу `ShopItem`. Затем пишем проверяем объекты на хэш.

Теперь проверим правильно ли работает метод `__hash__`. Для этого создаем несколько объектов класс `ShopItem`, затем вставим их тип данных множество(set). После этого в множество будет только два объекта, потому у двух объектов повторяются хэши из-за входящих данных.



methods.py

```
item1 = ShopItem("Кружка", 3, 500)
item2 = ShopItem("Ложка", 1, 200)
item3 = ShopItem("Кружка", 3, 500)

print({item1, item2, item3})
```

Задание

1. Реализуйте класс Book с атрибутами объекта title, author, publisher, year. Переопределите метод `__hash__` и `__eq__`, чтобы метод `__hash__` использовал только атрибуты title, author, publisher.

Контрольные вопросы

1. Почему используется тип данных tuple для хэширование, а не list
2. Почему важно переопределять метод `__hash__`, если переопределён `__eq__`?

Практическое задание № 14

Метод `__bool__`

Теоретическая часть

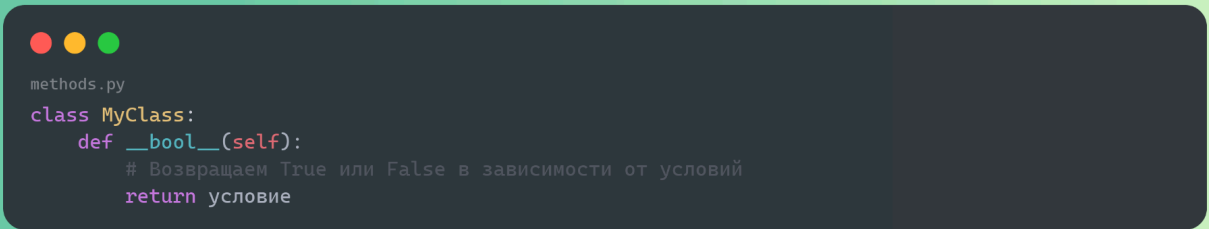
Метод `__bool__` в Python — это один из специальных методов, определяющих поведение объекта при преобразовании его в логическое значение (истинное или ложное).

Когда вызывается встроенная функция `bool(obj)`, Python пытается вызвать метод `__bool__`. Если этот метод не определён, вызывается метод `__len__`. Если оба метода отсутствуют, объект считается всегда истинным.

Общие правила:

1. `__bool__`: должен возвращать `True` или `False`. Обычно его переопределяют, чтобы указать, в каких условиях объект считается "истинным".
2. `__len__`: возвращает длину объекта. Если длина равна 0, о

Синтаксис:

A code editor window with a dark background and light green accents. It shows a Python class definition for MyClass with a __bool__ method. The method is commented to return True or False based on a condition.

```
methods.py
class MyClass:
    def __bool__(self):
        # Возвращаем True или False в зависимости от условий
        return условие
```

Пример:



methods.py

```
class CustomObject:
    def __init__(self, value):
        self.value = value

    def __bool__(self):
        return self.value > 0

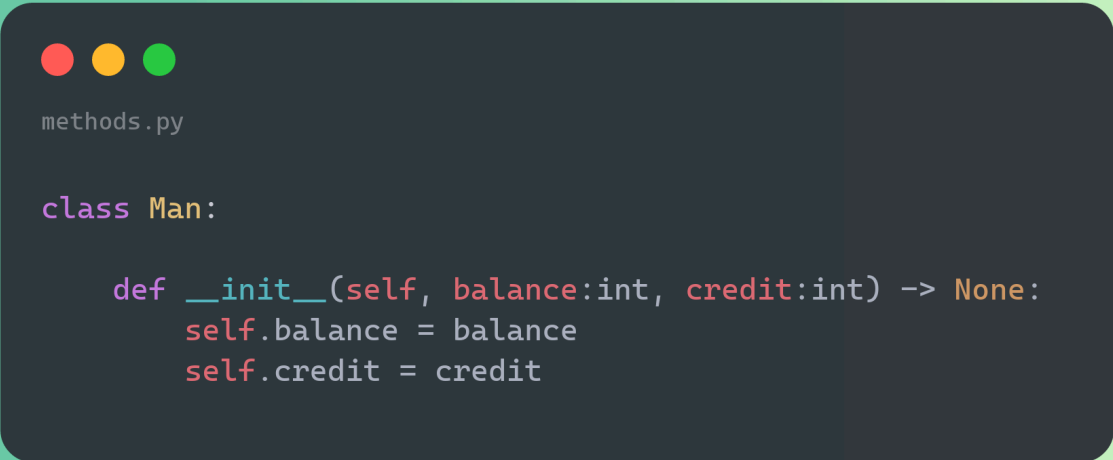
obj1 = CustomObject(5)
obj2 = CustomObject(-3)

print(bool(obj1))    # True
print(bool(obj2))    # False
```

Практическая часть

Реализуем класс Man, который будет в себе хранить два атрибута объекта balance и credit. Переопределим магический метод `__bool__`, чтобы при проверке объекта класса Man, выдавал True когда два атрибута положительные.

Создаем класс Man:

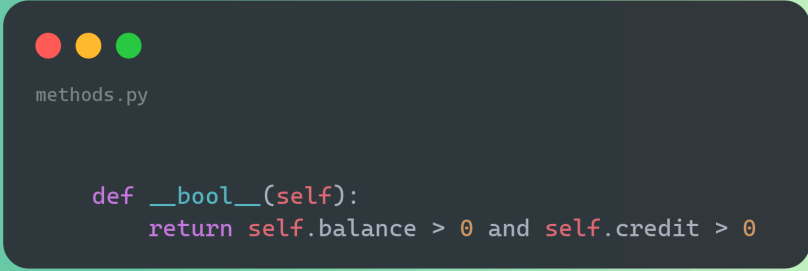


```
methods.py

class Man:

    def __init__(self, balance:int, credit:int) -> None:
        self.balance = balance
        self.credit = credit
```

Переопределяем метод `__bool__`:



```
methods.py

def __bool__(self):
    return self.balance > 0 and self.credit > 0
```

Теперь при проверке объекта с помощью функций `bool()`, будет проверяться значение атрибутов, которое было выставлено.

Задание

1. Создайте класс `Device` с атрибутами объекта класса `is_on` и `battery_level`. Реализуйте метод `__bool__`, чтобы он выдавал `True`, когда устройство включен и имеет зарядки больше 10%.

Контрольные вопросы

1. Как Python определяет истинность объекта, если метод `__bool__` отсутствует?
2. Чем метод `__bool__` отличается от `__len__`?
3. Как результат метода `__bool__` влияет на условные конструкции (`if`, `while` и т. д.)?

Практическое задание № 15

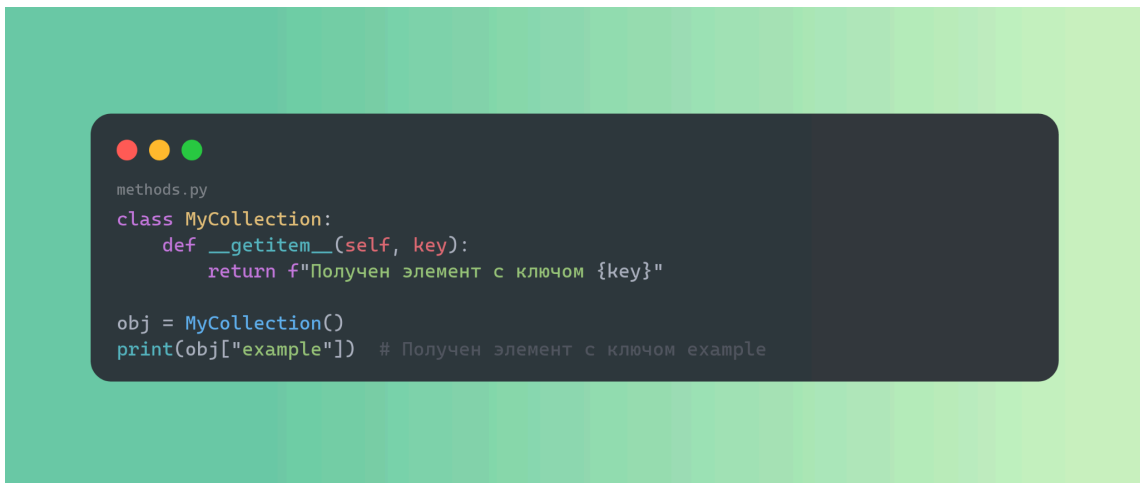
Методы `__getitem__`, `__setitem__` и `__delitem__`

Теоретическая часть

В Python специальные методы `__getitem__`, `__setitem__` и `__delitem__` используются для управления доступом, изменением и удалением элементов объекта так же, как мы работаем со словарями, списками или другими коллекциями.

1. `__getitem__(self, key)`

Этот метод вызывается, когда мы пытаемся получить элемент объекта с использованием ключа или индекса, как при обращении `obj[key]`.



2. `__setitem__(self, key, value)`

Этот метод вызывается, когда мы пытаемся установить значение для элемента с определённым ключом, как при записи `obj[key] = value`.



3. `__delitem__(self, key)`

Этот метод вызывается, когда мы пытаемся удалить элемент объекта по ключу, как при выполнении `del obj[key]`.

```
methods.py
class MyCollection:
    def __delitem__(self, key):
        print(f"Удалён элемент с ключом {key}")

obj = MyCollection()
del obj["example"] # Удалён элемент с ключом example
```

- Эти методы позволяют эмулировать поведение встроенных коллекций (например, списков, словарей).
- Их реализация полезна при создании собственных структур данных или обёрток над существующими коллекциями.

Практическая часть

Реализуем кастомный список лист, который будет выдавать последний элемент при выходе из диапазона, а также заменять и удалять.

Реализуем класс List:

```
methods.py
class List:
    def __init__(self, *data: list[any]):
        self.__data = list(data)
```

В инициализаторе используем звездочку, чтобы принимать все аргументы последовательно и без именований. Затем переопределяем метод `__getitem__`, который позволяет изменить обращение по индексам(если говорим про список, также можно изменить dict, set и т.д.).



methods.py

```
def __getitem__(self, index):  
    if index > len(self.__data):  
        return self.__data[-1]  
    return self.__data[index]
```

В данном методе прописали логику, который выводить последний элемент при выходе из диапазона в большую сторону. Теперь реализуем аналогичным способом магические методы `__setitem__` и `__delitem__`.



methods.py

```
def __setitem__(self, index, value):  
    if index > len(self.__data):  
        self.__data[-1] = value  
        return self.__data  
    self.__data[index] = value  
    return self.__data  
  
def __delitem__(self, index):  
    if index > len(self.__data):  
        del self.__data[-1]  
    del self.__data[index]
```

Задание

1. Реализуйте класс `KeyValueStore`, который будет аналогом словаря, но при каждом изменении будет логировать их в txt файл.

Контрольные вопросы

1. Для чего используются методы `__getitem__`, `__setitem__` и `__delitem__` в Python?
2. Что происходит, если вызвать `obj[key]` для объекта, у которого реализован только метод `__getitem__`?

Практическое задание № 16

Методы `__iter__` и `__next__`

Теоретическая часть

Методы `__iter__` и `__next__` играют ключевую роль в создании итераторов в Python. Эти методы позволяют объекту быть "итерируемым", то есть таким, который можно перебрать с помощью цикла `for` или других инструментов, работающих с итераторами.

Итераторы и итерируемые объекты

1. **Итерируемый объект** — это объект, который может возвращать итератор. Он должен реализовать метод `__iter__`.
2. **Итератор** — это объект, который реализует метод `__next__`, возвращающий следующий элемент последовательности. Когда элементы заканчиваются, `__next__` должен выбросить исключение `StopIteration`.

Метод `__iter__`

- Должен возвращать объект, который реализует метод `__next__`.
- Обычно, объект возвращает самого себя, если он уже является итератором.

Метод `__next__`

- Вызывается для получения следующего элемента последовательности.
- Если элементов больше нет, метод выбрасывает исключение `StopIteration`.

Пример работы итератора



methods.py

```
class Counter:
    def __init__(self, start, end):
        self.current = start
        self.end = end

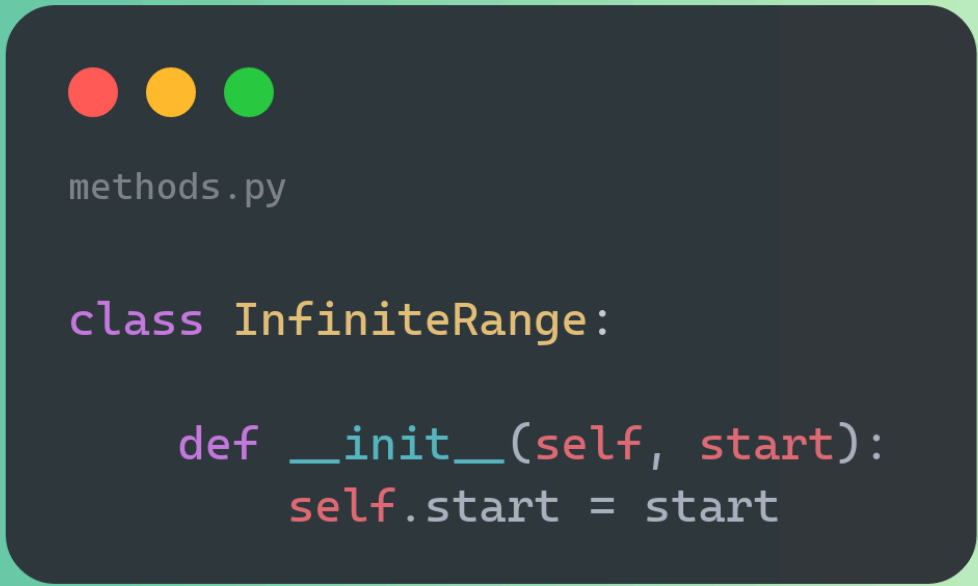
    def __iter__(self):
        return self

    def __next__(self):
        if self.current ≥ self.end:
            raise StopIteration
        self.current += 1
        return self.current - 1
```

Практическая часть

В этой практической реализуем класс `InfiniteRange`, который будет принимать только стартовую точку, а затем идти по бесконечности.

Создаем класс `InfiniteRange` с атрибутом объекта `start`:

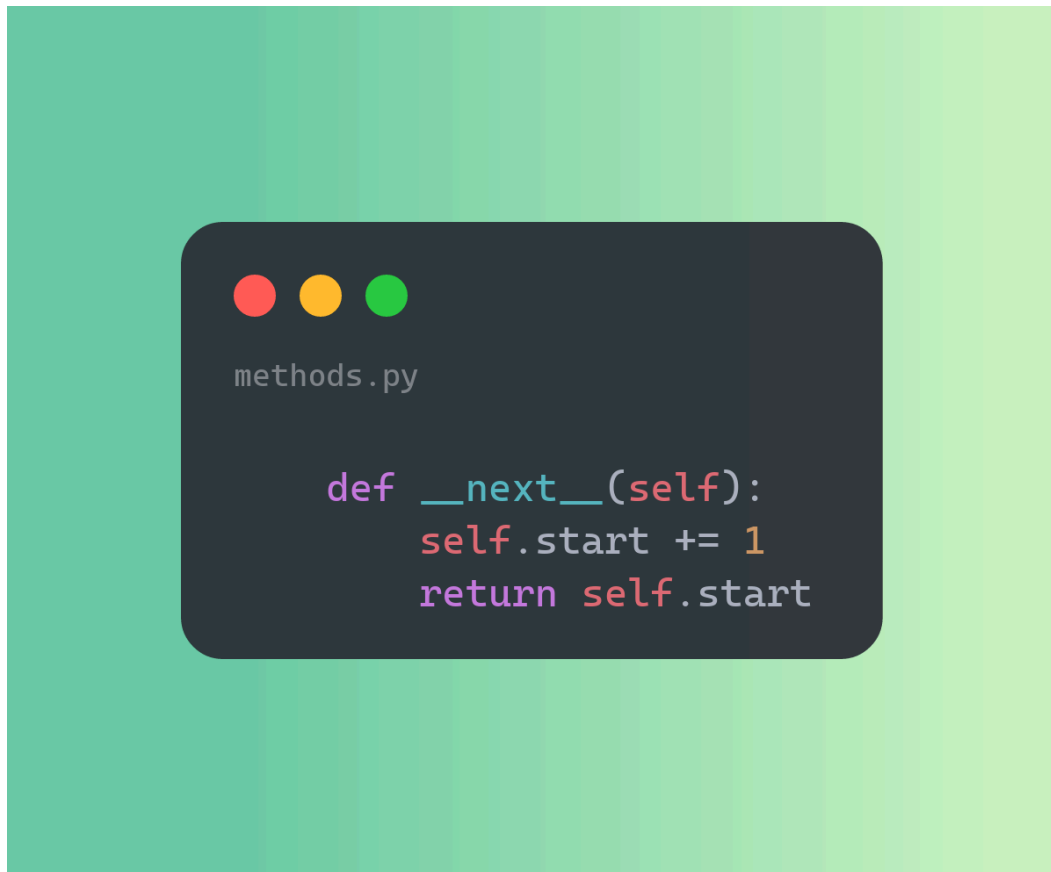


methods.py

```
class InfiniteRange:

    def __init__(self, start):
        self.start = start
```

Переопределяем метод `__next__`, который отвечает за определенный шаг итерации(такой же шаг как в классе `range`).



Теперь при использовании класса `InfiniteRange` будет выводиться ошибка “object is not iterable”, которая возникает потому не переопределен метод `__iter__`. Метод `__iter__` нужен, чтобы инициализировать наш итератор.

Например, нам нужно создать цикл, который будет выводить цифры от 0 до 10. Для этого создаем переменную, которая будет находиться вне цикла – это и будет наша метод `__iter__`, а метод `__next__` будет самим циклом с условием.



Данной практической наш итератор был бесконечным, но при созданных кастомных итераторах они обычно заканчиваются, Чтобы закончить итератор используют исключение `StopIteration`.

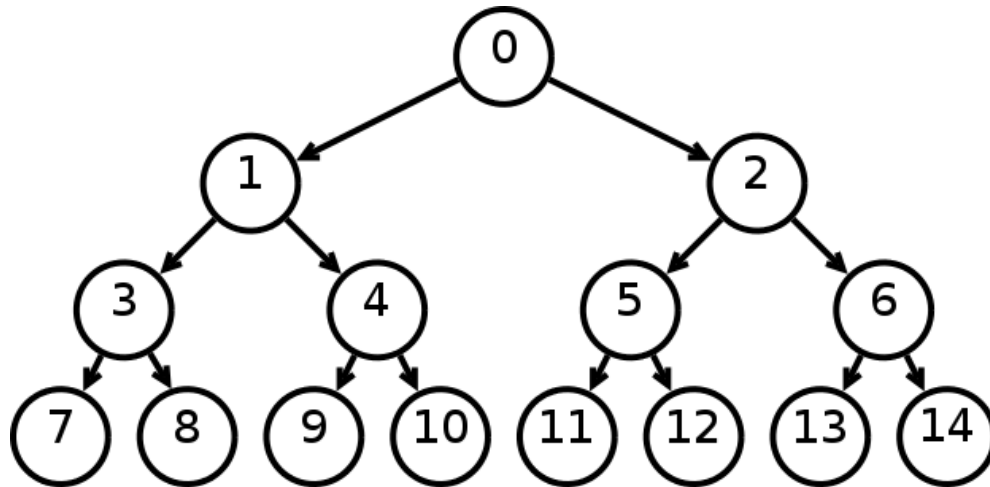
Задание

1. Создайте итератор, который возвращает числа Фибоначчи до заданного значения.
2. Создайте бесконечный итератор, который генерирует числа, кратные заданному числу.

Контрольные вопросы

1. Что такое итератор в Python?
2. Чем итерируемый объект отличается от итератора?
3. Как остановить итерацию внутри метода `__next__`?

Самостоятельная работа №1



Бинарное дерево — это структура, в которой каждый узел имеет не более двух дочерних узлов, называемых левым и правым. Узлы располагаются так, что:

- Левое поддерево каждого узла содержит только узлы со значениями, меньшими значения узла.
- Правое поддерево содержит только узлы с большими значениями.

Материал для ознакомления:

📺 КАК РАБОТАЮТ ДЕРЕВЬЯ | СТРУКТУРЫ ДАННЫХ

Задание

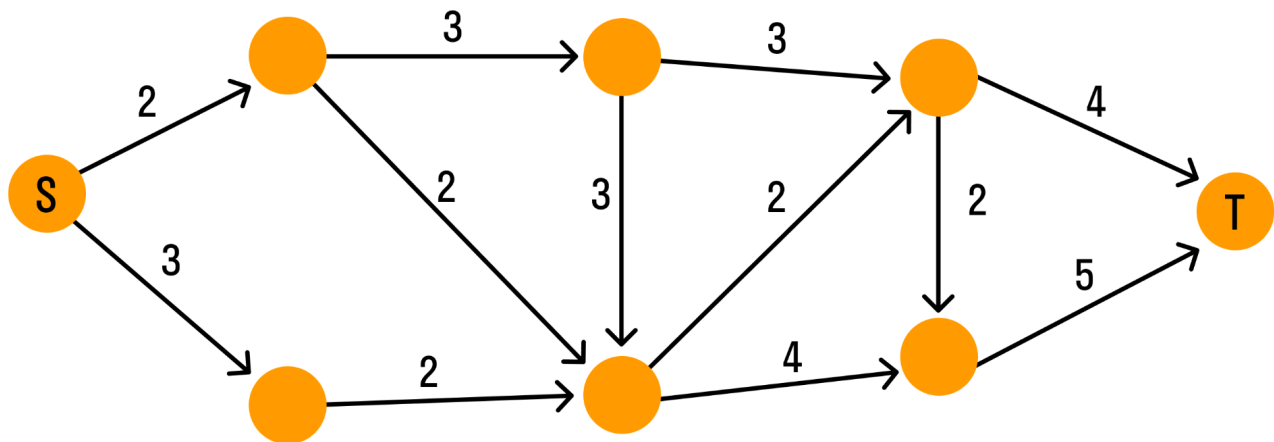
Реализовать структуру узла и бинарного дерева.

- Определите класс **Node**, который будет представлять узел бинарного дерева.
- Определите класс **BinaryTree**, который будет содержать методы для вставки, поиска и удаления узлов.

Реализовать методы:

- **Вставка узла:** Метод `insert(self, value)`, который добавляет новый узел с заданным значением.
- **Поиск узла:** Метод `search(self, value)`, который ищет узел с заданным значением и возвращает его, если он найден.
- **Удаление узла:** Метод `delete(self, value)`, который удаляет узел с заданным значением.

Самостоятельная работа №2



Граф — это математическая структура, состоящая из множества **вершин** (или узлов) и множества **рёбер**, соединяющих пары вершин. Графы широко используются в различных областях, включая компьютерные науки, социологию, биологию и экономику, для моделирования различных систем и процессов.

Основные типы графов:

1. **Ненаправленные графы:** Рёбра не имеют направления. Если существует ребро между вершинами AAA и BBB, то можно переходить от AAA к BBB и от BBB к AAA.
2. **Направленные графы:** Рёбра имеют направление. Если существует ребро от AAA к BBB, можно переходить только от AAA к BBB.
3. **Взвешенные графы:** Каждое ребро имеет вес (или стоимость), что позволяет учитывать расстояния или затраты на переход между вершинами.
4. **Смешанные графы:** Содержат как направленные, так и ненаправленные рёбра.
5. **Циклы и деревья:**
 - **Цикл:** Замкнутая последовательность рёбер, в которой каждая вершина посещается ровно один раз.
 - **Дерево:** Связный граф без циклов.

Материал для ознакомления:

📺 КАК РАБОТАЮТ ГРАФЫ | СТРУКТУРЫ ДАННЫХ

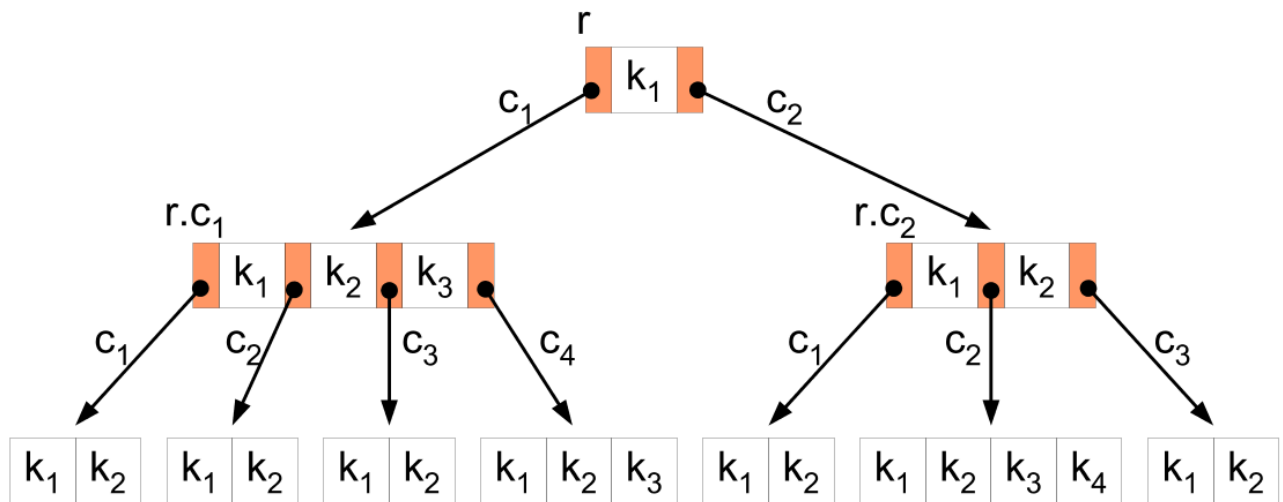
Задание:

Создайте класс `Graph`, который будет представлять граф с использованием списка смежности.

Реализуйте методы:

- `add_edge(u, v)` — для добавления рёбер.
- `display()` — для отображения графа.
- `dfs(start)` — для выполнения поиска в глубину и вывода посещённых вершин.
- `bfs(start)` — для выполнения поиска в ширину и вывода посещённых вершин.

Самостоятельная работа №3



В-деревья — это самообновляющаяся структура данных, которая поддерживает сортированные данные и позволяет эффективно выполнять операции вставки, удаления и поиска. Они часто используются в базах данных и файловых системах благодаря своей способности сохранять данные на диске, минимизируя количество операций ввода-вывода.

Применения В-деревьев:

- Используются в базах данных для индексации данных и ускорения операций поиска.
- Применяются в файловых системах для управления файлами и директориями.
- Широко используются в алгоритмах, требующих эффективного доступа к большим объемам данных.

Материал для ознакомления:

📺 [Как устроен B-TREE индекс в базах данных](#)

📺 [Базы данных B-tree](#)

Контрольные вопросы:

1. Что такое В-дерево и для чего оно используется?
2. Как происходит поиск в В-дереве?
3. Что происходит при вставке нового ключа в переполненный узел?